Data 260

Homework 6

Name: Kshitija Shinde
SJSU ID: 019146491

Value	**Definition**
SID4	6491
PORT_BASE	8000 + (6491 mod 900) = 8000 + 191 = 8191
PREFIX	"s" + 6491 = s6491
SEED	6491
VERIFY_SEED	260000 + 6491 = 266491
DOMAIN_ID	6491 mod 8 = 3
Assigned Domain	Grocery supply and recall notices

Hardware: MacBook Air, 13-inch, M4, 2025, Apple M4 chip, 16GB RAM, macOS Tahoe 26.5.2

Local model used: qwen3:8b via Ollama

Tagged commit hash: 32f75f59209bb2a2fd1bf8f19b96b522f4cd64b7

Repository: https://github.com/Kshitija-Shinde9/data260-6491

Part 1 - MongoDB Document API

This homework builds on HW5. Add a MongoDB-backed document view of your domain.

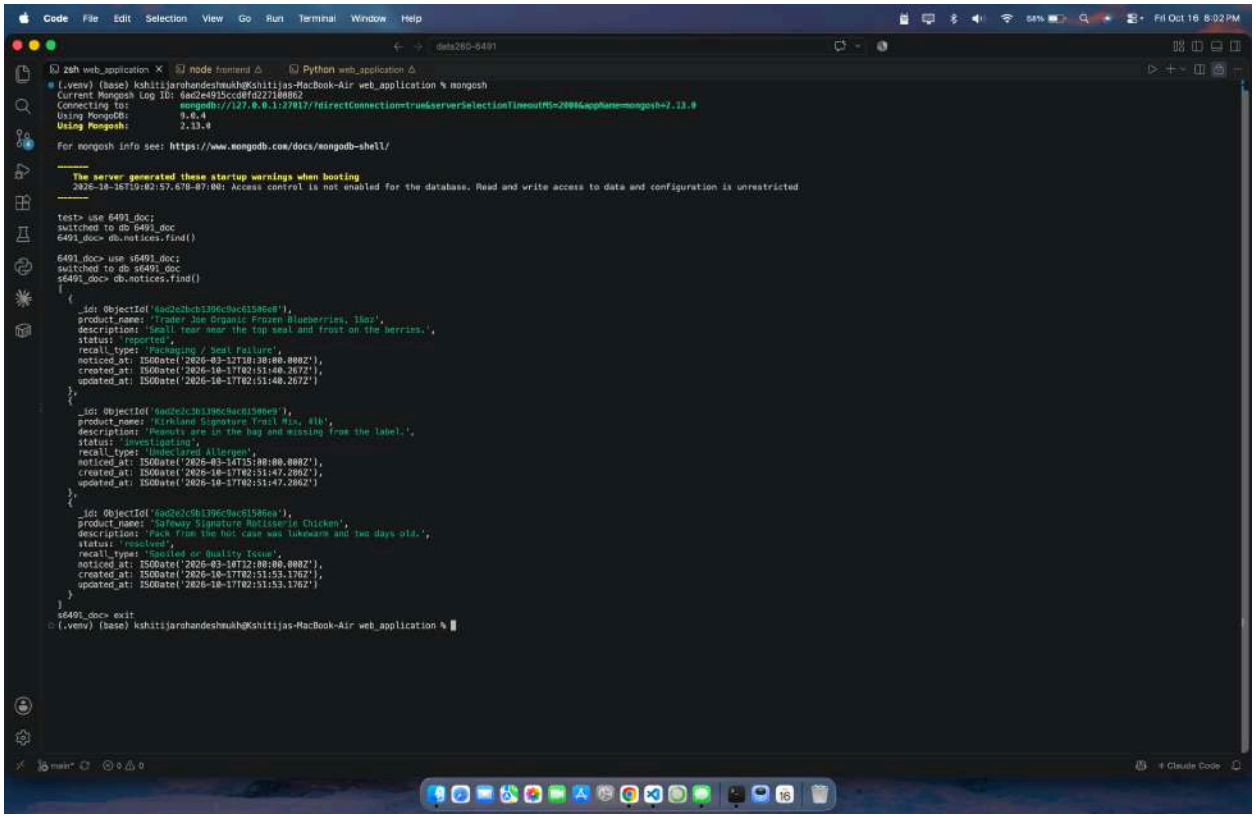
I. Database

- Create the MongoDB database _doc with a collection for a documentoriented representation of your domain (e.g., for a task-management-style setup this would be a "tasks" collection - map the fields below to your own domain entity instead).
- Include at least: one required short text field (max 100 characters), one optional description field, two constrained categorical fields (e.g., a status-style enum with a default, and a category-style field with a fixed set of options), one required date/time field, and automatic created/updated timestamps.
- The schema must include proper validation rules such as required fields, string length limits, and enum constraints.

```
DB_NAME = "s6491_doc"
COLLECTION_NAME = "notices"

NOTICE_SCHEMA = {
   "bsonType": "object",
   "required": [
       "product_name",
       "status",
       "recall_type",
       "noticed_at",
       "created_at",
       "updated_at",
   ],
   "properties": {
       "product_name": {"bsonType": "string", "minLength": 1,
"maxLength": 100},
       "description": {"bsonType": ["string", "null"], "maxLength":
2000},
       "status": {"enum": ["reported", "investigating", "resolved"]},
       "recall_type": {
           "enum": [
               "Packaging / Seal Failure",
               "Contamination",
```

```
                "Undeclared Allergen",
                "Spoiled or Quality Issue",
            ]
        },
        "noticed_at": {"bsonType": "date"},
        "created_at": {"bsonType": "date"},
        "updated_at": {"bsonType": "date"},
    },
}
```



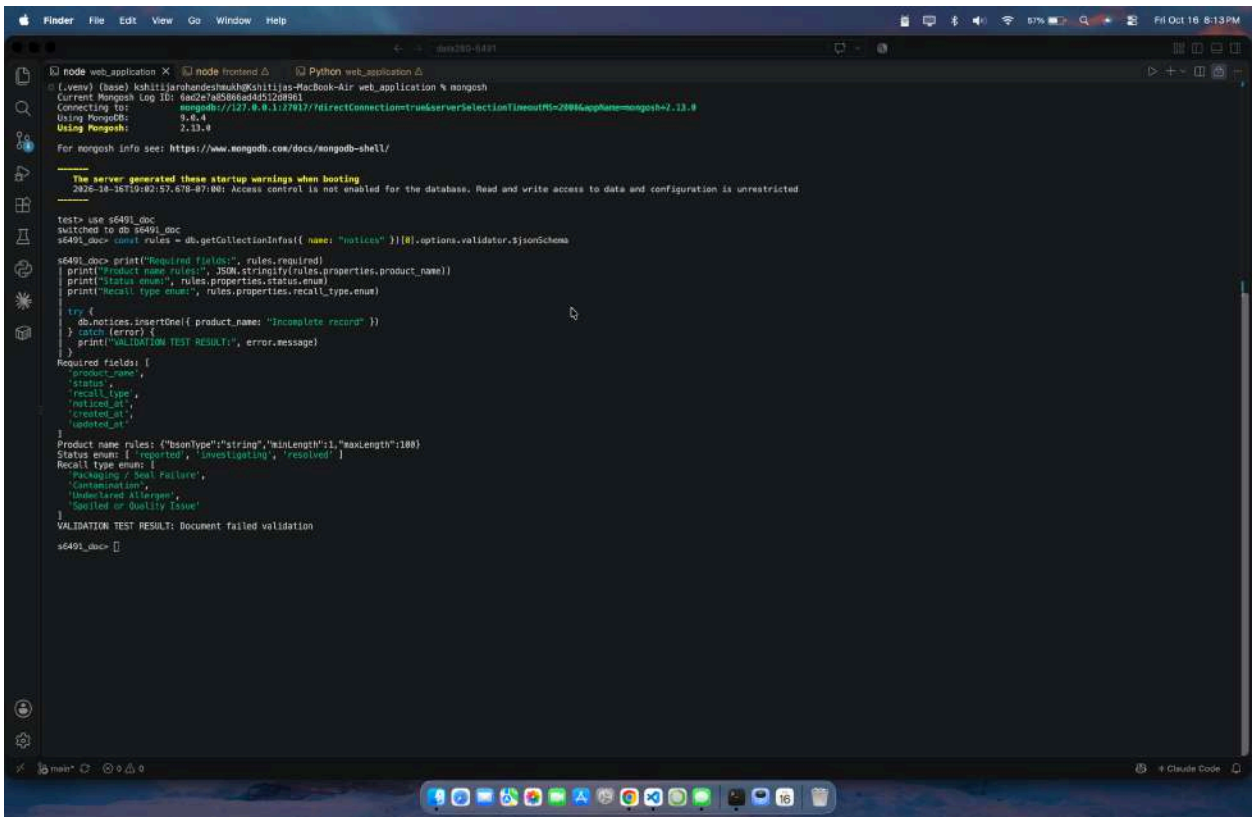
```
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % mongosh
Current Mongosh Log ID: 6a02e4915ccd0fd227108862
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.13.0
Using MongoDB:          9.0.4
Using Mongosh:          2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-16T19:02:57.678-07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use 6491_doc;
switched to db 6491_doc
6491_doc> db.notices.find()

6491_doc> use s6491_doc;
switched to db s6491_doc
s6491_doc> db.notices.find()
[
  {
    _id: ObjectId('6ad2e2bcb1396c9ac61506e8'),
    product_name: 'Trader Joe Organic Frozen Blueberries, 16oz',
    description: 'Small tear near the top seal and frost on the berries.',
    status: 'reported',
    recall_type: 'Packaging / Seal Failure',
    noticed_at: ISODate('2026-03-12T10:30:00.000Z'),
    created_at: ISODate('2026-10-17T02:51:40.267Z'),
    updated_at: ISODate('2026-10-17T02:51:40.267Z')
  },
  {
    _id: ObjectId('6ad2e2c3b1396c9ac61506e9'),
    product_name: 'Kirkland Signature Trail Mix, 41lb',
    description: 'Peanuts are in the bag and missing from the label.',
    status: 'investigating',
    recall_type: 'Undeclared Allergen',
    noticed_at: ISODate('2026-03-14T15:00:00.000Z'),
    created_at: ISODate('2026-10-17T02:51:47.206Z'),
    updated_at: ISODate('2026-10-17T02:51:47.206Z')
  },
  {
    _id: ObjectId('6ad2e2c9b1396c9ac61506ea'),
    product_name: 'Safeway Signature Rotisserie Chicken',
    description: 'Pack from the hot case was lukewarm and two days old.',
    status: 'resolved',
    recall_type: 'Spoiled or Quality Issue',
    noticed_at: ISODate('2026-03-10T12:00:00.000Z'),
    created_at: ISODate('2026-10-17T02:51:53.176Z'),
    updated_at: ISODate('2026-10-17T02:51:53.176Z')
  }
]
s6491_doc> exit
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application %
```



```
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % mongosh
Current Mongosh Log ID: 6ad2e7a85866ad4d5120f961
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.13.0
Using MongoDB:          9.0.4
Using Mongosh:          2.13.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-16T19:02:57.678-07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use s6491_doc
switched to db s6491_doc
s6491_doc> const rules = db.getCollectionInfos({ name: "notices" })[0].options.validator.$jsonSchema

s6491_doc> print("Required fields:", rules.required)
| print("Product name rules:", JSON.stringify(rules.properties.product_name))
| print("Status enum:", rules.properties.status.enum)
| print("Recall type enum:", rules.properties.recall_type.enum)
|
| try {
|   db.notices.insertOne({ product_name: "Incomplete record" })
| } catch (error) {
|   print("VALIDATION TEST RESULT:", error.message)
| }
Required fields: [
  'product_name',
  'status',
  'recall_type',
  'noticed_at',
  'created_at',
  'updated_at'
]
Product name rules: {"bsonType":"string","minLength":1,"maxLength":100}
Status enum: [ 'reported', 'investigating', 'resolved' ]
Recall type enum: [
  'Packaging / Seal Failure',
  'Contamination',
  'Undeclared Allergen',
  'Spoiled or Quality Issue'
]
VALIDATION TEST RESULT: Document failed validation

s6491_doc>
```

II. API Implementation

- Implement a REST API using FastAPI with complete CRUD operations: create (POST), retrieve all (GET), get a single record by ID (GET /:id), update (PUT /:id), delete (DELETE /:id).
- Include proper error handling with appropriate HTTP status codes for scenarios like invalid data, resource not found, and server errors.
- Use middleware for CORS and JSON parsing for request bodies. Structure your code with separate files for server configuration, PyMongo database connection, data models/schemas (Pydantic), and route handlers.

```
app.add_middleware(
   CORSMiddleware,
   allow_origins=["http://localhost:5173"],
   allow_credentials=True,
   allow_methods=["*"],
   allow_headers=["*"],
)

app.add_middleware(JsonBodyMiddleware)

app.include_router(auth_router)
app.include_router(api_auth_router)
app.include_router(suppliers_router)
app.include_router(recalls_router)
app.include_router(bench_router)
app.include_router(notices_router)
```

```
router = APIRouter(prefix="/api/notices", tags=["notices"])

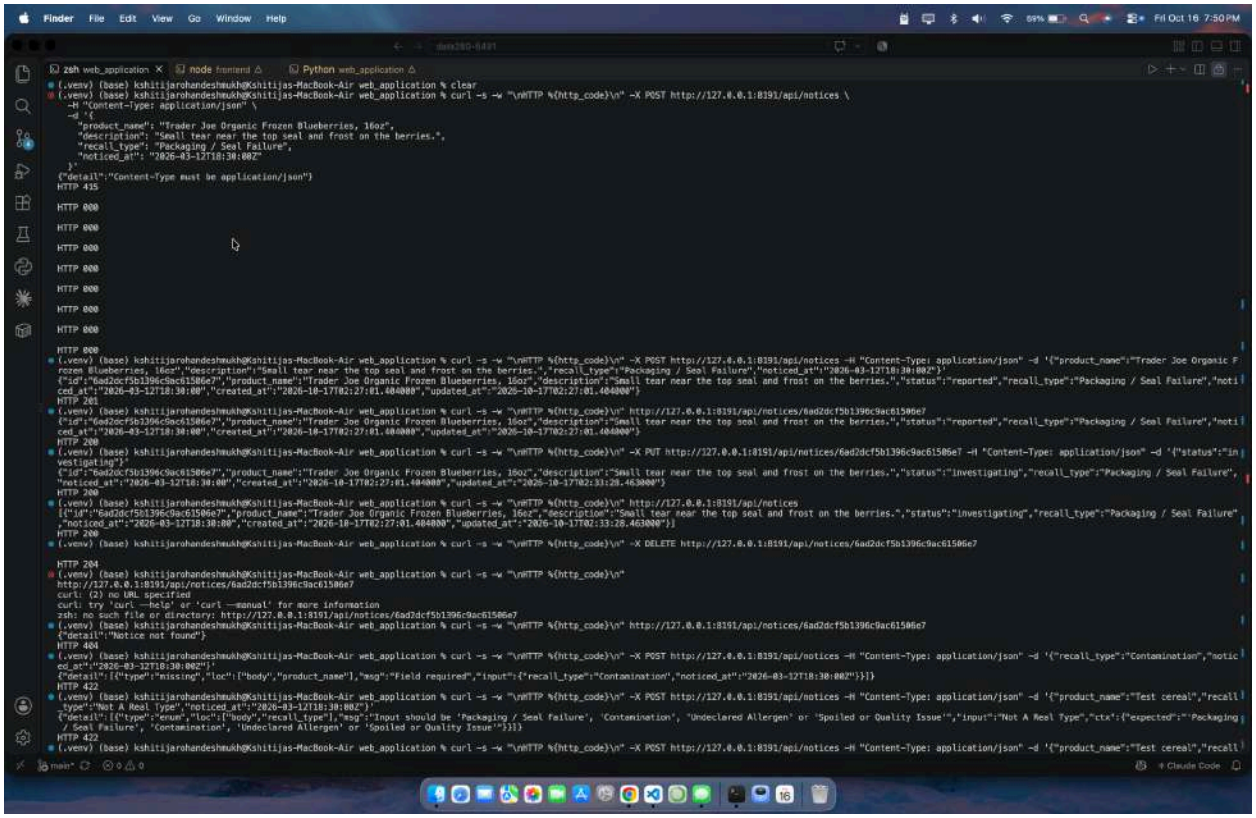
@router.post("", response_model=NoticeOut, status_code=201)
def create_notice(body: NoticeCreate): ...

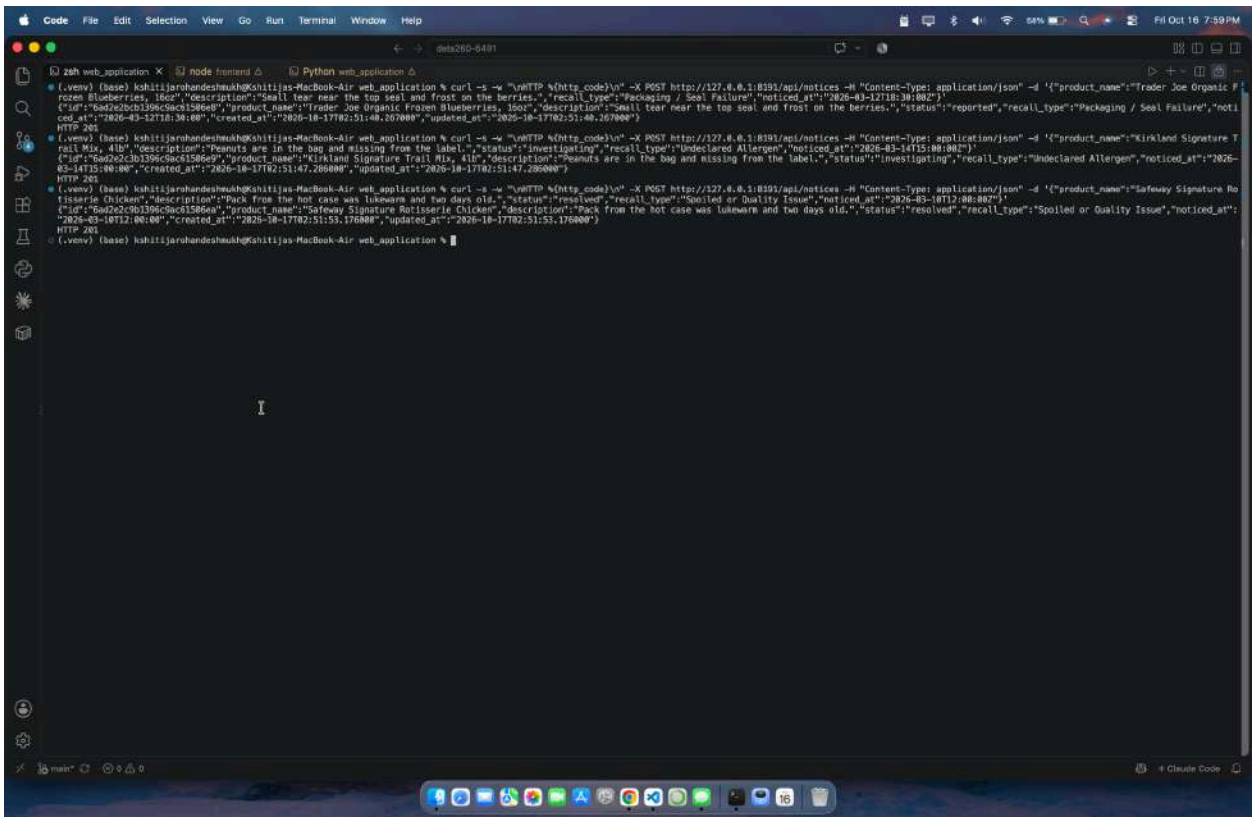
@router.get("", response_model=List[NoticeOut])
def list_notices(): ...

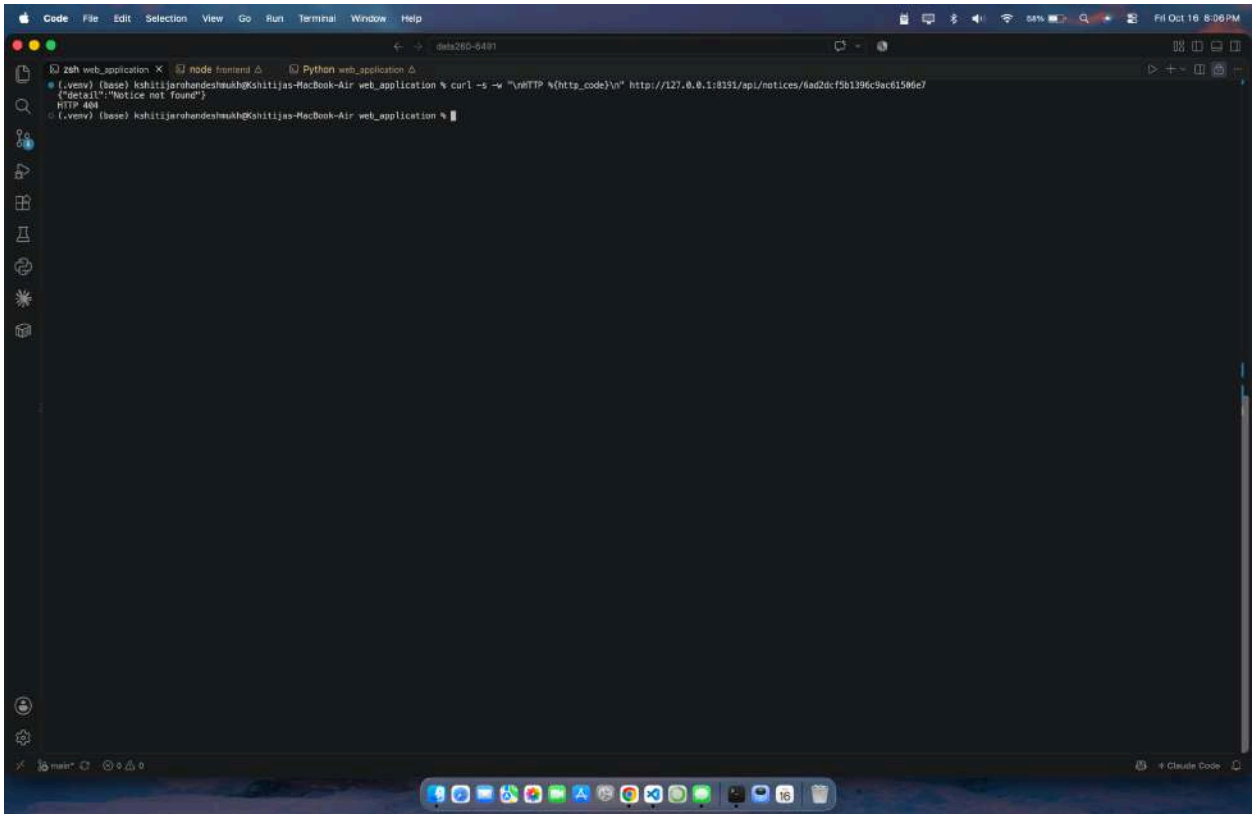
@router.get("/{notice_id}", response_model=NoticeOut)
def get_notice(notice_id: str): ...
```

```
@router.put("/{notice_id}", response_model=NoticeOut)
def update_notice(notice_id: str, body: NoticeUpdate): ...

@router.delete("/{notice_id}", status_code=204)
def delete_notice(notice_id: str): ...
```


Finder File Edit View Go Window Help
Fri Oct 16 7:50 PM
zsh web_application
node frontend
Python web_application
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % clear
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices \
-H "Content-Type: application/json" \
-d '{
"product_name": "Trader Joe Organic Frozen Blueberries, 16oz",
"description": "Small tear near the top seal and frost on the berries.",
"recall_type": "Packaging / Seal Failure",
"noticed_at": "2026-03-12T18:30:00Z"
}'
{"detail":"Content-Type must be application/json"}
HTTP 415
HTTP 000
HTTP 000
HTTP 000
HTTP 000
HTTP 000
HTTP 000
HTTP 000
HTTP 000
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00Z"}'
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"reported","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:27:01.404000"}
HTTP 201
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"reported","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:27:01.404000"}
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X PUT http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7 -H "Content-Type: application/json" -d '{"status":"investigating"}'
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"investigating","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:33:28.463000"}
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices
[{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"investigating","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:33:28.463000"}]
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X DELETE http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
HTTP 204
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n"
http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
curl: (2) no URL specified
curl: try 'curl --help' or 'curl --manual' for more information
zsh: no such file or directory: http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
{"detail":"Notice not found"}
HTTP 404
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"recall_type":"Contamination","noticed_at":"2026-03-12T18:30:00Z"}'
{"detail":[{"type":"missing","loc":["body","product_name"],"msg":"Field required","input":{"recall_type":"Contamination","noticed_at":"2026-03-12T18:30:00Z"}}]}
HTTP 422
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Test cereal","recall_type":"Not A Real Type","noticed_at":"2026-03-12T18:30:00Z"}'
{"detail":[{"type":"enum","loc":["body","recall_type"],"msg":"Input should be 'Packaging / Seal Failure', 'Contamination', 'Undeclared Allergen' or 'Spoiled or Quality Issue'","input":"Not A Real Type","ctx":{"expected":"'Packaging / Seal Failure', 'Contamination', 'Undeclared Allergen' or 'Spoiled or Quality Issue'"}}]}
HTTP 422
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Test cereal","recall

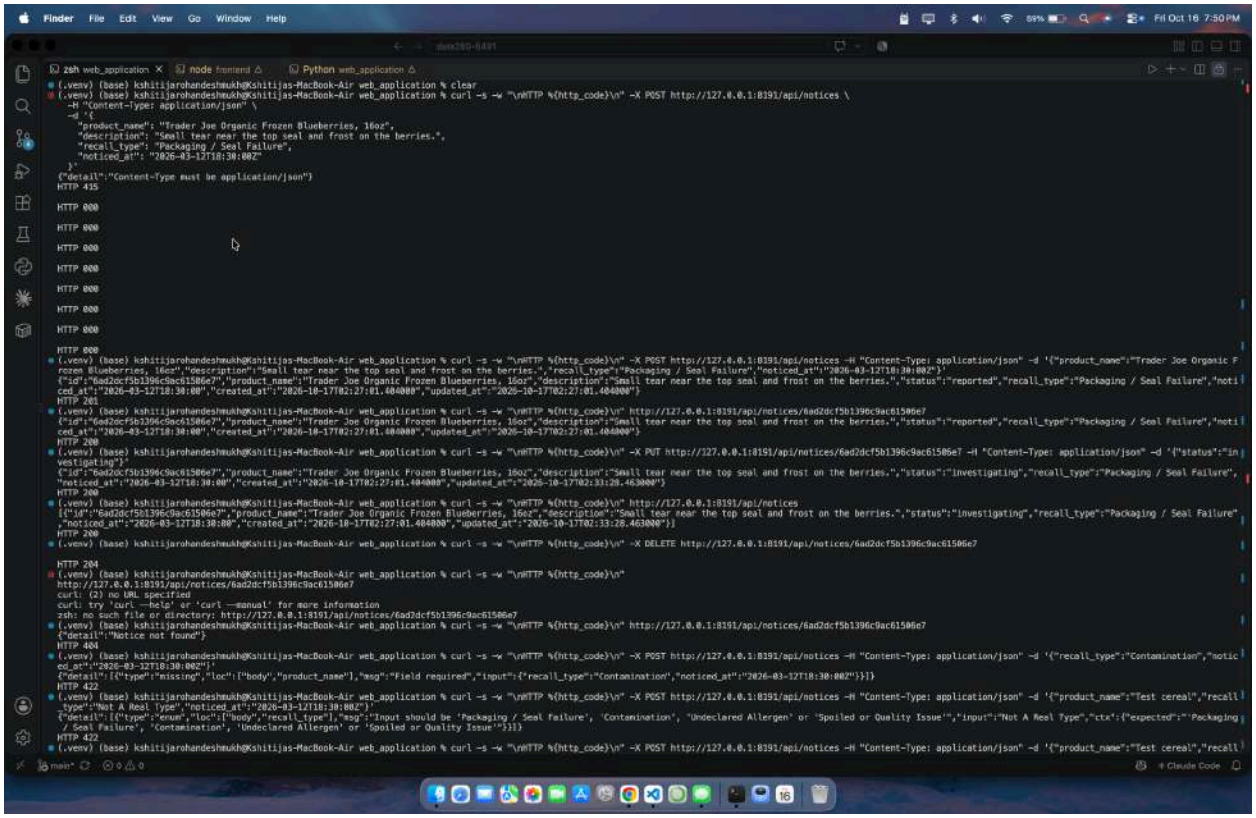

Code File Edit Selection View Go Run Terminal Window Help
Fri Oct 16 7:59 PM
zsh web_application
node frontend
Python web_application
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top sesl and frost on the berries.","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00Z"}'
{"id":"6ad2e2bcb1396c9ac61506e8","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"reported","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:51:40.267000","updated_at":"2026-10-17T02:51:40.267000"}
HTTP 201
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Kirkland Signature Trail Mix, 41b","description":"Peanuts are in the bag and missing from the label.","status":"investigating","recall_type":"Undeclared Allergen","noticed_at":"2026-03-14T15:00:00Z"}'
{"id":"6ad2e2c3b1396c9ac61506e9","product_name":"Kirkland Signature Trail Mix, 41b","description":"Peanuts are in the bag and missing from the label.","status":"investigating","recall_type":"Undeclared Allergen","noticed_at":"2026-03-14T15:00:00","created_at":"2026-10-17T02:51:47.286000","updated_at":"2026-10-17T02:51:47.286000"}
HTTP 201
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Safeway Signature Rotisserie Chicken","description":"Pack from the hot case was lukewarm and two days old.","status":"resolved","recall_type":"Spoiled or Quality Issue","noticed_at":"2026-03-10T12:00:00Z"}'
{"id":"6ad2e2c9b1396c9ac61506ea","product_name":"Safeway Signature Rotisserie Chicken","description":"Pack from the hot case was lukewarm and two days old.","status":"resolved","recall_type":"Spoiled or Quality Issue","noticed_at":"2026-03-10T12:00:00","created_at":"2026-10-17T02:51:53.176000","updated_at":"2026-10-17T02:51:53.176000"}
HTTP 201
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application %

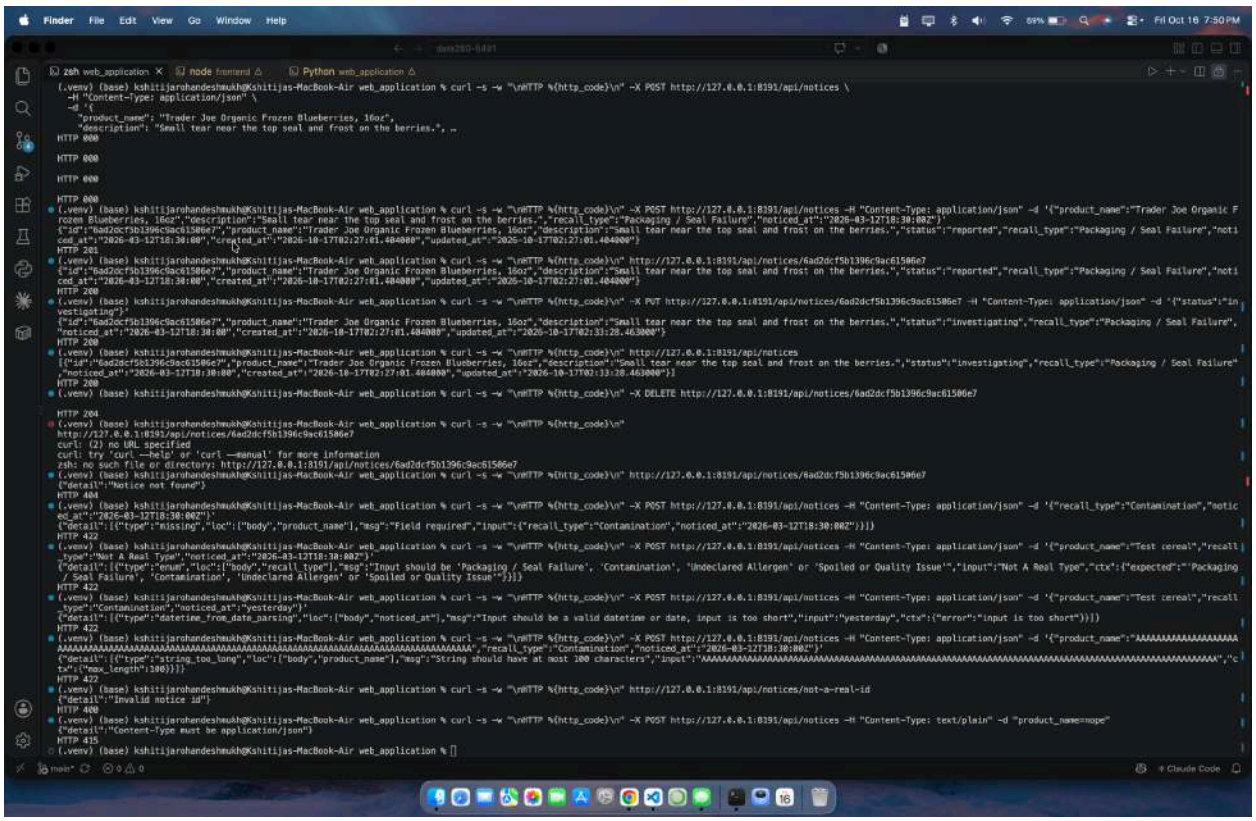

Code File Edit Selection View Go Run Terminal Window Help
Fri Oct 16 8:06 PM
zsh web_application
node frontend
Python web_application
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
{"detail":"Notice not found"}
HTTP 404
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application %

III. Data Validation

- Implement validation using Pydantic schemas to enforce data integrity including required fields, string lengths, enum values, and proper date formats.
- Ensure that validation errors return clear error messages with appropriate HTTP status codes.
- Test validation by attempting to create records with invalid data such as missing required fields, invalid enum values, and incorrect data types.

```
class NoticeCreate(BaseModel):
    product_name: str = Field(min_length=1, max_length=100)
    description: Optional[str] = Field(default=None, max_length=2000)
    status: NoticeStatus = NoticeStatus.reported
    recall_type: RecallType
    noticed_at: datetime
```




Finder File Edit View Go Window Help
Fri Oct 16 7:50 PM
zsh web_application
node frontend
Python web_application
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices \
-H "Content-Type: application/json" \
-d '{
"product_name": "Trader Joe Organic Frozen Blueberries, 16oz",
"description": "Small tear near the top seal and frost on the berries.", …
HTTP 000
HTTP 000
HTTP 000
HTTP 000
HTTP 000
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00Z"}'
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"reported","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:27:01.404000"}
HTTP 201
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"reported","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:27:01.404000"}
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X PUT http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7 -H "Content-Type: application/json" -d '{"status":"investigating"}'
{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"investigating","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:33:28.463000"}
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices
[{"id":"6ad2dcf5b1396c9ac61506e7","product_name":"Trader Joe Organic Frozen Blueberries, 16oz","description":"Small tear near the top seal and frost on the berries.","status":"investigating","recall_type":"Packaging / Seal Failure","noticed_at":"2026-03-12T18:30:00","created_at":"2026-10-17T02:27:01.404000","updated_at":"2026-10-17T02:33:28.463000"}]
HTTP 200
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X DELETE http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
HTTP 204
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n"
http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
curl: (2) no URL specified
curl: try 'curl --help' or 'curl --manual' for more information
zsh: no such file or directory: http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" http://127.0.0.1:8191/api/notices/6ad2dcf5b1396c9ac61506e7
{"detail":"Notice not found"}
HTTP 404
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"recall_type":"Contamination","noticed_at":"2026-03-12T18:30:00Z"}'
{"detail":[{"type":"missing","loc":["body","product_name"],"msg":"Field required","input":{"recall_type":"Contamination","noticed_at":"2026-03-12T18:30:00Z"}}]}
HTTP 422
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Test cereal","recall_type":"Not A Real Type","noticed_at":"2026-03-12T18:30:00Z"}'
{"detail":[{"type":"enum","loc":["body","recall_type"],"msg":"Input should be 'Packaging / Seal Failure', 'Contamination', 'Undeclared Allergen' or 'Spoiled or Quality Issue'","input":"Not A Real Type","ctx":{"expected":"'Packaging / Seal Failure', 'Contamination', 'Undeclared Allergen' or 'Spoiled or Quality Issue'"}}]}
HTTP 422
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"Test cereal","recall_type":"Contamination","noticed_at":"yesterday"}'
{"detail":[{"type":"datetime_from_date_parsing","loc":["body","noticed_at"],"msg":"Input should be a valid datetime or date, input is too short","input":"yesterday","ctx":{"error":"input is too short"}}]}
HTTP 422
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8191/api/notices -H "Content-Type: application/json" -d '{"product_name":"AAAA

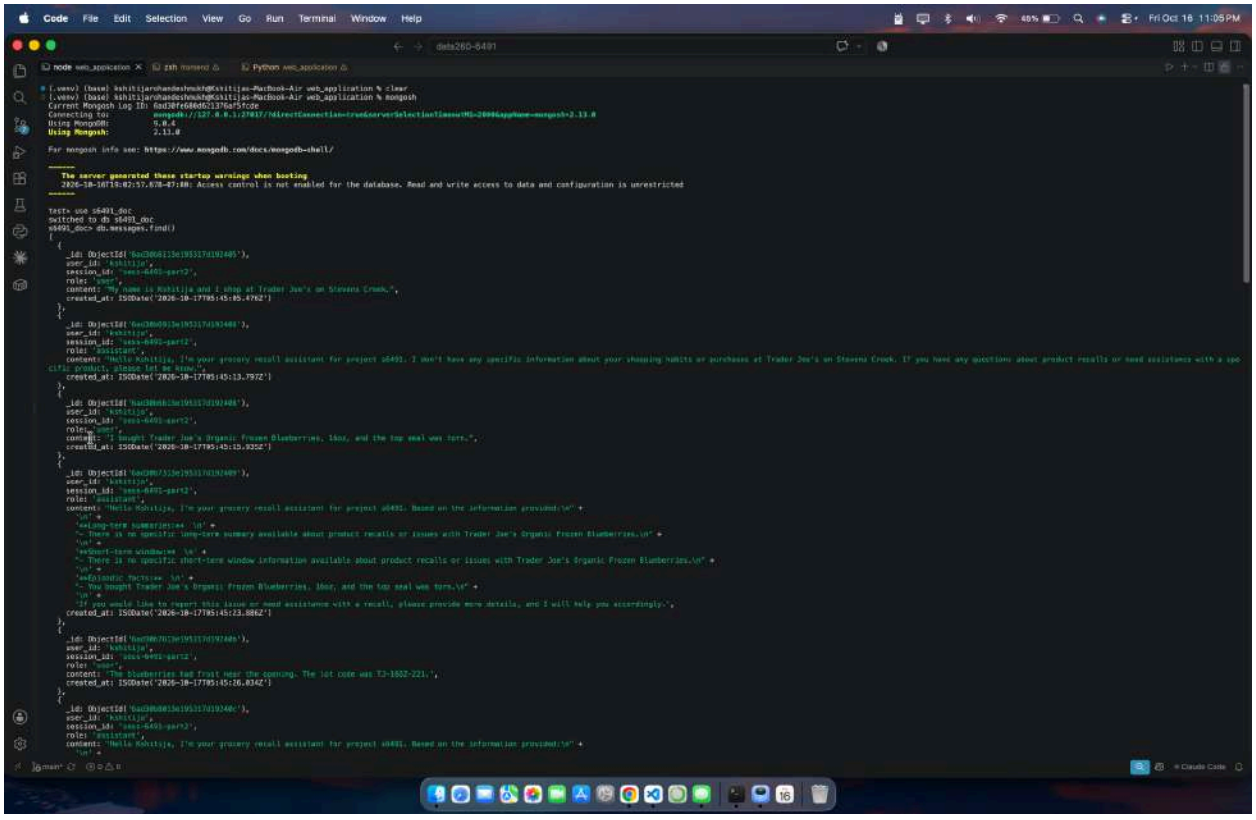
Part 2 - Short-, Long-, and Episodic Memory

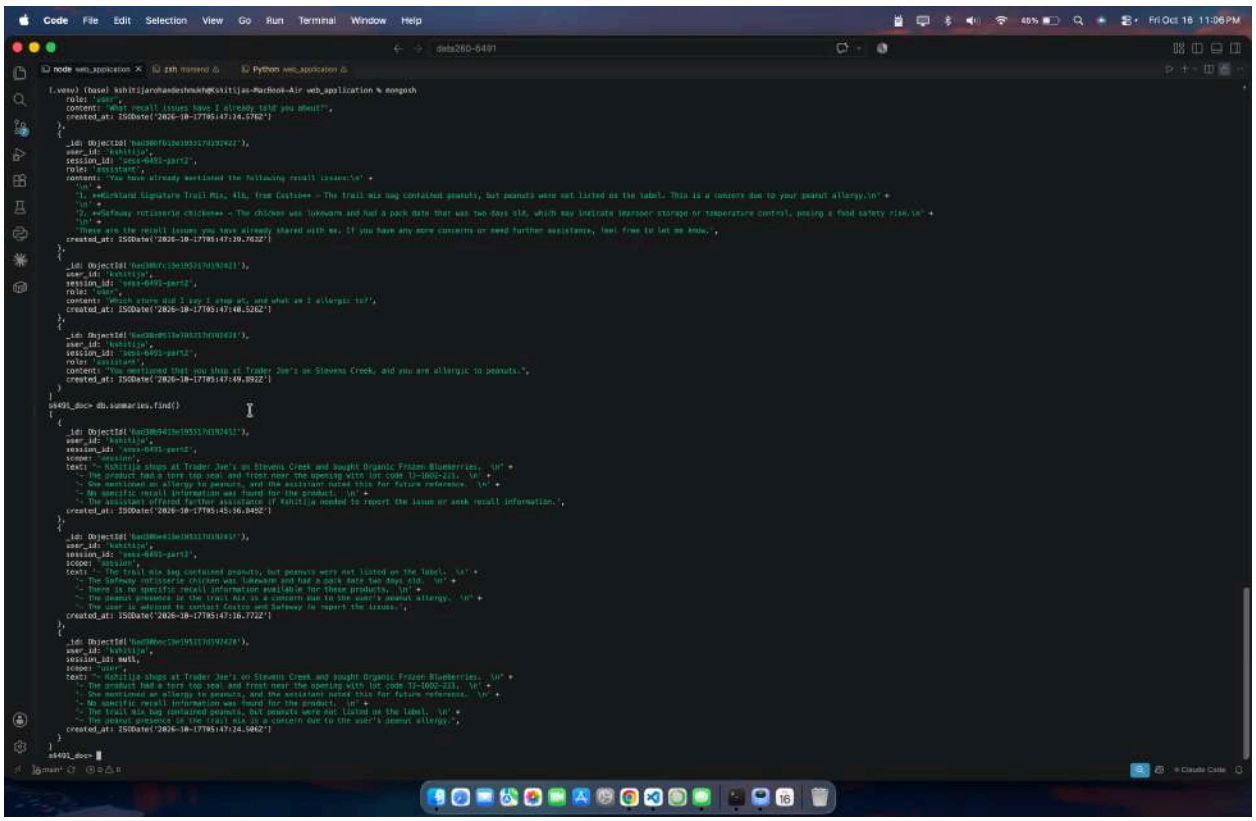
Use the same MongoDB database with new collections: messages, summaries, episodes. Keep your Part 1 collection unchanged - you are only reusing the same database/connection.

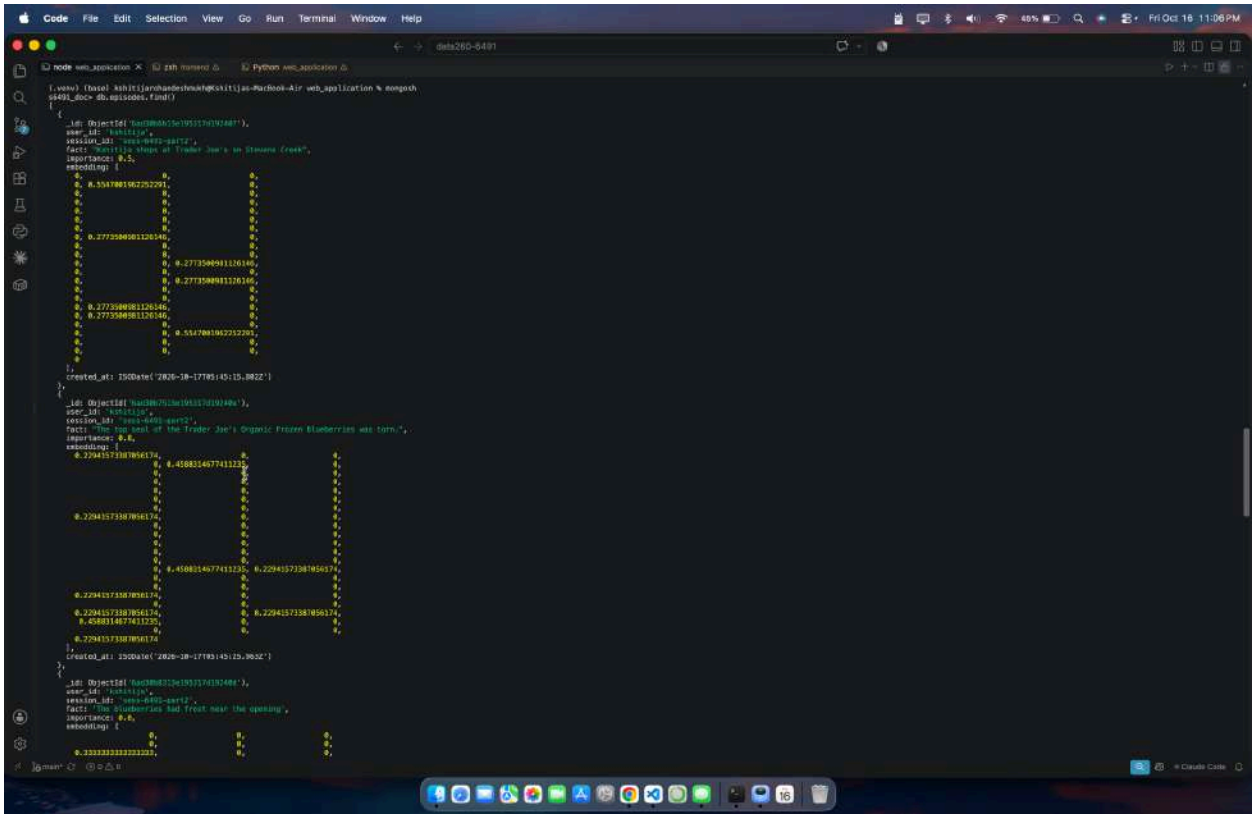
MongoDB collections (reuse the same DB)

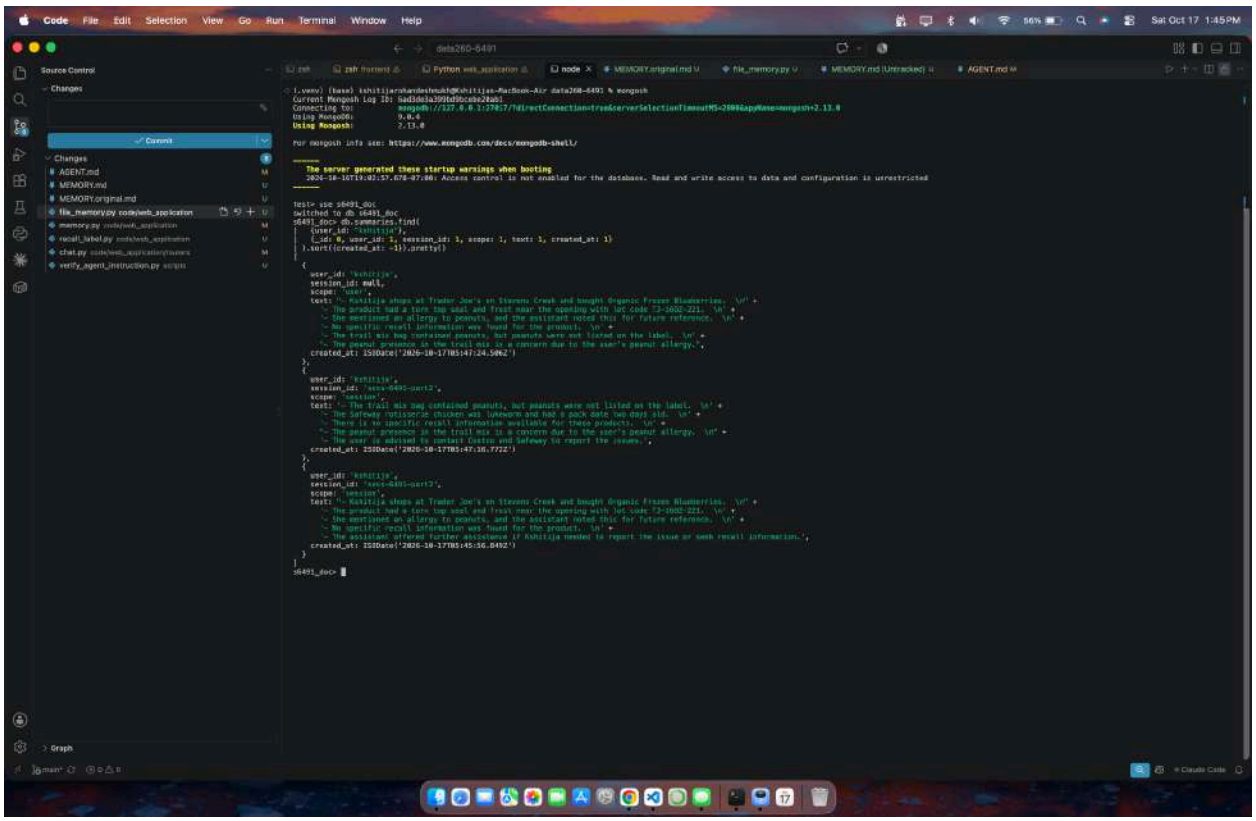
Create new collections: (3 points)
Messages
Fields: user_id, session_id, role (user/assistant), content, created_at.
Summaries
Fields: user_id, session_id (nullable for lifetime), scope ('session' | 'user'), text, created_at.
Episodes
Fields: user_id, session_id, fact (short text), importance (0..1), embedding (array of floats), created_at.

```
def ensure_memory_collections() -> None:
    _ensure_collection("messages", MESSAGE_SCHEMA)
    _ensure_collection("summaries", SUMMARY_SCHEMA)
    _ensure_collection("episodes", EPISODE_SCHEMA)
```









FastAPI Endpoints

1. POST /api/chat
   - Request: { user_id, session_id (optional), message }
   - Server steps: save the user message to messages; load the short-term window (last N messages, N = SHORT_TERM_N); load the latest session summary and lifetime summary from summaries; retrieve the top-k relevant episodic facts; compose the prompt (system primer + long-term summaries + short-term window + current message + episodic facts); call the model through your HW1 model adapter (local Ollama model); save the assistant reply to messages; every SUMMARIZE_EVERY_USER_MSGS user messages, generate/update a session summary, and occasionally condense session summaries into a lifetime summary.
   - Response: the assistant reply plus an object showing what was used - short-term message count/list, the long-term summary text (if any), and the episodic facts retrieved for this turn.

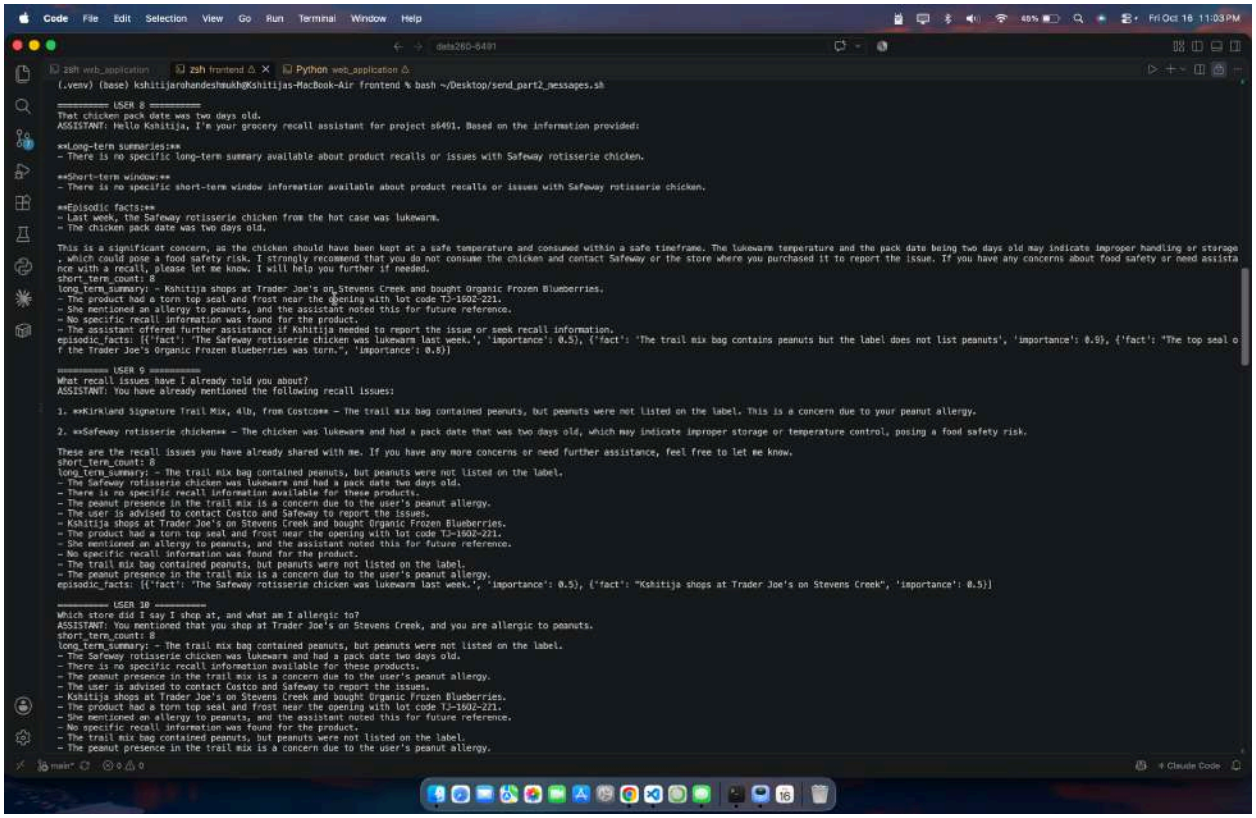
2. GET /api/memory/{user_id} - returns the last ~16 messages, latest session summary, latest lifetime summary, and last ~20 episodic facts.

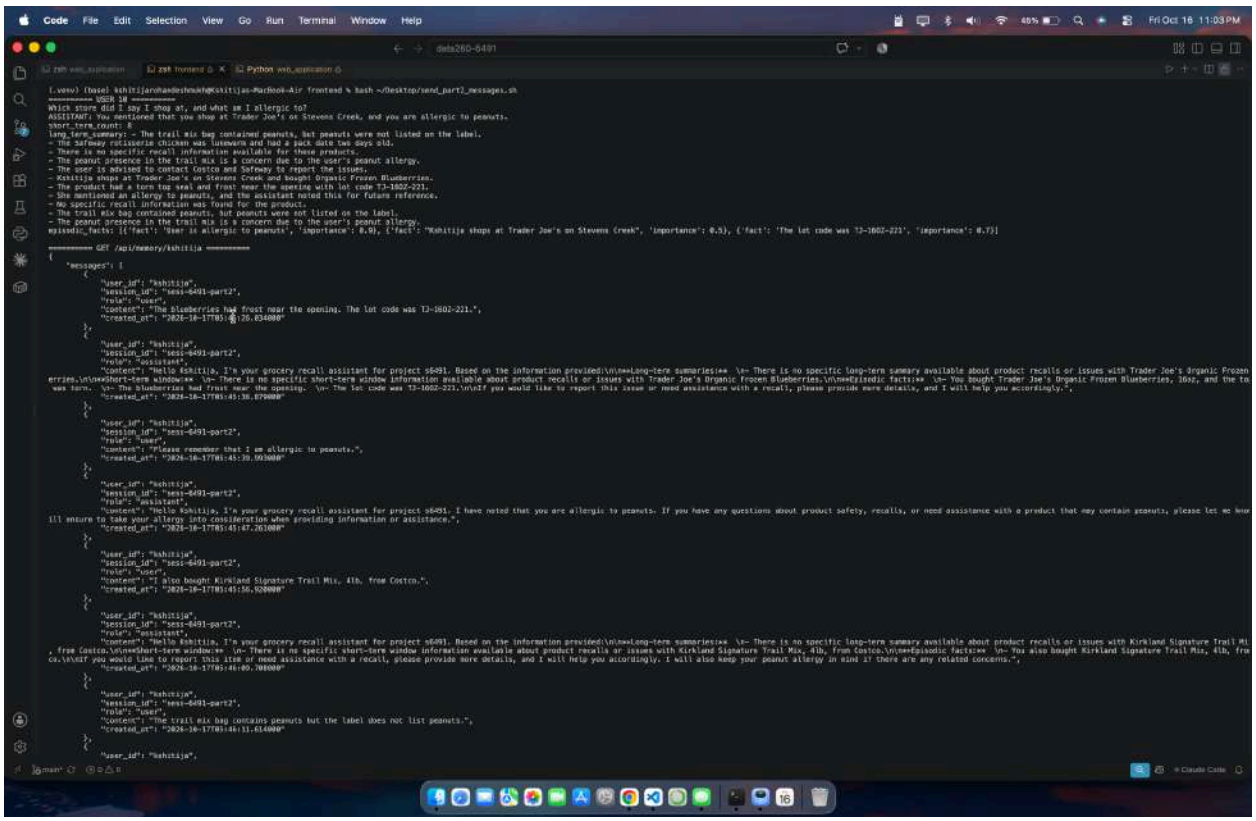
3. GET /api/aggregate/{user_id} - returns daily message counts (grouped by date) and a small list of the most recent summaries.

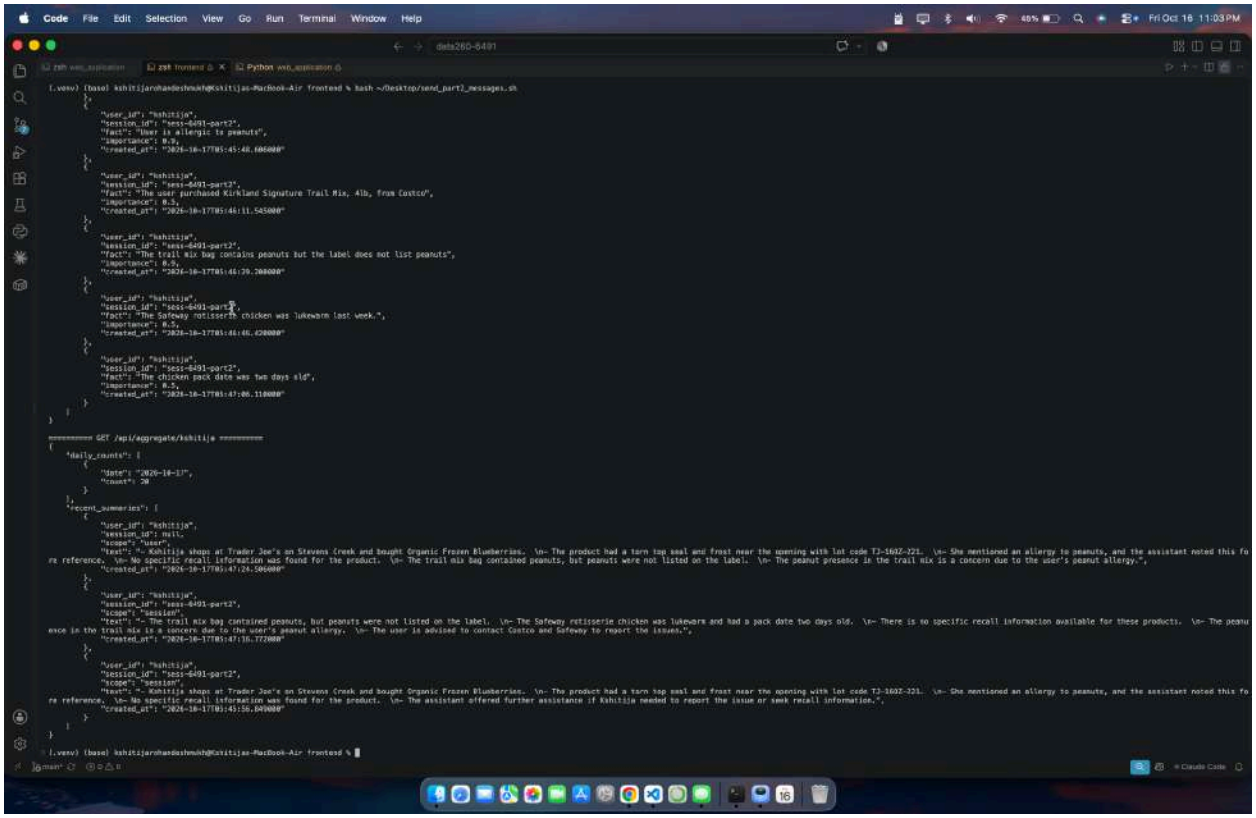
```
@router.post("/chat", response_model=ChatOut)
def post_chat(body: ChatIn):
   session_id = body.session_id or f"sess-{uuid.uuid4().hex[:8]}"
   return run_chat_turn(body.user_id, session_id, body.message.strip())

@router.get("/memory/{user_id}", response_model=MemoryOut)
def get_memory(user_id: str):
   return memory_view(user_id)

@router.get("/aggregate/{user_id}", response_model=AggregateOut)
def get_aggregate(user_id: str):
   return aggregate_view(user_id)
```


Code File Edit Selection View Go Run Terminal Window Help
Fri Oct 16 11:03 PM
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air frontend % bash ~/Desktop/send_part2_messages.sh
========== USER 8 ==========
That chicken pack date was two days old.
ASSISTANT: Hello Kshitija, I'm your grocery recall assistant for project s6491. Based on the information provided:
Long-term summaries:
- There is no specific long-term summary available about product recalls or issues with Safeway rotisserie chicken.
Short-term window:
- There is no specific short-term window information available about product recalls or issues with Safeway rotisserie chicken.
Episodic facts:
- Last week, the Safeway rotisserie chicken from the hot case was lukewarm.
- The chicken pack date was two days old.
This is a significant concern, as the chicken should have been kept at a safe temperature and consumed within a safe timeframe. The lukewarm temperature and the pack date being two days old may indicate improper handling or storage, which could pose a food safety risk. I strongly recommend that you do not consume the chicken and contact Safeway or the store where you purchased it to report the issue. If you have any concerns about food safety or need assistance with a recall, please let me know. I will help you further if needed.
short_term_count: 8
long_term_summary: - Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The assistant offered further assistance if Kshitija needed to report the issue or seek recall information.
episodic_facts: [{'fact': 'The Safeway rotisserie chicken was lukewarm last week.', 'importance': 0.5}, {'fact': 'The trail mix bag contains peanuts but the label does not list peanuts', 'importance': 0.9}, {'fact': 'The top seal of the Trader Joe's Organic Frozen Blueberries was torn.', 'importance': 0.8}]
========== USER 9 ==========
What recall issues have I already told you about?
ASSISTANT: You have already mentioned the following recall issues:
1. **Kirkland Signature Trail Mix, 4lb, from Costco** - The trail mix bag contained peanuts, but peanuts were not listed on the label. This is a concern due to your peanut allergy.
2. **Safeway rotisserie chicken** - The chicken was lukewarm and had a pack date that was two days old, which may indicate improper storage or temperature control, posing a food safety risk.
These are the recall issues you have already shared with me. If you have any more concerns or need further assistance, feel free to let me know.
short_term_count: 8
long_term_summary: - The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The Safeway rotisserie chicken was lukewarm and had a pack date two days old.
- There is no specific recall information available for these products.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
- The user is advised to contact Costco and Safeway to report the issues.
- Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
episodic_facts: [{'fact': 'The Safeway rotisserie chicken was lukewarm last week.', 'importance': 0.5}, {'fact': "Kshitija shops at Trader Joe's on Stevens Creek", 'importance': 0.5}]
========== USER 10 ==========
Which store did I say I shop at, and what am I allergic to?
ASSISTANT: You mentioned that you shop at Trader Joe's on Stevens Creek, and you are allergic to peanuts.
short_term_count: 8
long_term_summary: - The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The Safeway rotisserie chicken was lukewarm and had a pack date two days old.
- There is no specific recall information available for these products.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
- The user is advised to contact Costco and Safeway to report the issues.
- Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.


Code File Edit Selection View Go Run Terminal Window Help
Fri Oct 16 11:03 PM
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air frontend % bash ~/Desktop/send_part2_messages.sh
========== USER 10 ==========
Which store did I say I shop at, and what am I allergic to?
ASSISTANT: You mentioned that you shop at Trader Joe's on Stevens Creek, and you are allergic to peanuts.
short_term_count: 8
long_term_summary: - The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The Safeway rotisserie chicken was lukewarm and had a pack date two days old.
- There is no specific recall information available for these products.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
- The user is advised to contact Costco and Safeway to report the issues.
- Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
episodic_facts: [{'fact': 'User is allergic to peanuts', 'importance': 0.9}, {'fact': "Kshitija shops at Trader Joe's on Stevens Creek", 'importance': 0.5}, {'fact': 'The lot code was TJ-1602-221', 'importance': 0.7}]
========== GET /api/memory/kshitija ==========
{
"messages": [
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "user",
"content": "The blueberries had frost near the opening. The lot code was TJ-1602-221.",
"created_at": "2026-10-17T01:45:26.634000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "assistant",
"content": "Hello Kshitija, I'm your grocery recall assistant for project s6491. Based on the information provided:\n\n**Long-term summaries:** \n- There is no specific long-term summary available about product recalls or issues with Trader Joe's Organic Frozen Berries.\n\n**Short-term window:** \n- There is no specific short-term window information available about product recalls or issues with Trader Joe's Organic Frozen Blueberries.\n\n**Episodic facts:** \n- You bought Trader Joe's Organic Frozen Blueberries, 16oz, and the top seal was torn. \n- The blueberries had frost near the opening. \n- The lot code was TJ-1602-221.\n\nIf you would like to report this issue or need assistance with a recall, please provide more details, and I will help you accordingly.",
"created_at": "2026-10-17T01:45:36.679000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "user",
"content": "Please remember that I am allergic to peanuts.",
"created_at": "2026-10-17T01:45:39.093000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "assistant",
"content": "Hello Kshitija, I'm your grocery recall assistant for project s6491. I have noted that you are allergic to peanuts. If you have any questions about product safety, recalls, or need assistance with a product that may contain peanuts, please let me know. I'll ensure to take your allergy into consideration when providing information or assistance.",
"created_at": "2026-10-17T01:45:47.261000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "user",
"content": "I also bought Kirkland Signature Trail Mix, 4lb, from Costco.",
"created_at": "2026-10-17T01:45:56.928000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "assistant",
"content": "Hello Kshitija, I'm your grocery recall assistant for project s6491. Based on the information provided:\n\n**Long-term summaries:** \n- There is no specific long-term summary available about product recalls or issues with Kirkland Signature Trail Mix, 4lb, from Costco.\n\n**Short-term window:** \n- There is no specific short-term window information available about product recalls or issues with Kirkland Signature Trail Mix, 4lb, from Costco.\n\n**Episodic facts:** \n- You also bought Kirkland Signature Trail Mix, 4lb, from Costco.\n\nIf you would like to report this item or need assistance with a recall, please provide more details, and I will help you accordingly. I will also keep your peanut allergy in mind if there are any related concerns.",
"created_at": "2026-10-17T01:46:09.708000"
},
{
"user_id": "kshitija",
"session_id": "sess-6491-part2",
"role": "user",
"content": "The trail mix bag contains peanuts but the label does not list peanuts.",
"created_at": "2026-10-17T01:46:11.614000"
},
{
"user_id": "kshitija",


(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air frontend % bash ~/Desktop/send_part2_messages.sh
========== GET /api/aggregate/kshitija ==========
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air frontend %

Memory logic

- Short-term memory: sliding window of the last N messages for the session.
- Long-term summaries: trigger every M user messages; summarize recent conversation into concise bullets under scope='session'; build/refresh a lifetime summary by condensing multiple session summaries under scope='user' with session_id = null.
- Episodic memory: extract up to 3 short facts per user message with an importance score (0..1); embed each fact and store the array in MongoDB; for each turn, retrieve the top-k relevant episodes by cosine similarity between the current message embedding and stored episodes.

```
SHORT_TERM_N = int(os.getenv("SHORT_TERM_N", "8"))
SUMMARIZE_EVERY_USER_MSGS = int(os.getenv("SUMMARIZE_EVERY_USER_MSGS", "4"))
EPISODE_TOP_K = int(os.getenv("EPISODE_TOP_K", "3"))

def load_short_term(session_id: str, limit: int) -> list:
   rows = list(get_messages().find({"session_id": session_id}).sort("created_at",
-1).limit(limit))
   rows.reverse()
   return rows

def maybe_summarize(user_id: str, session_id: str, client) -> None:
   user_turns = get_messages().count_documents({"session_id": session_id, "role":
"user"})
   if user_turns == 0 or user_turns % SUMMARIZE_EVERY_USER_MSGS != 0:
   return
# write scope="session"; every second session summary, write scope="user" with
session_id=None

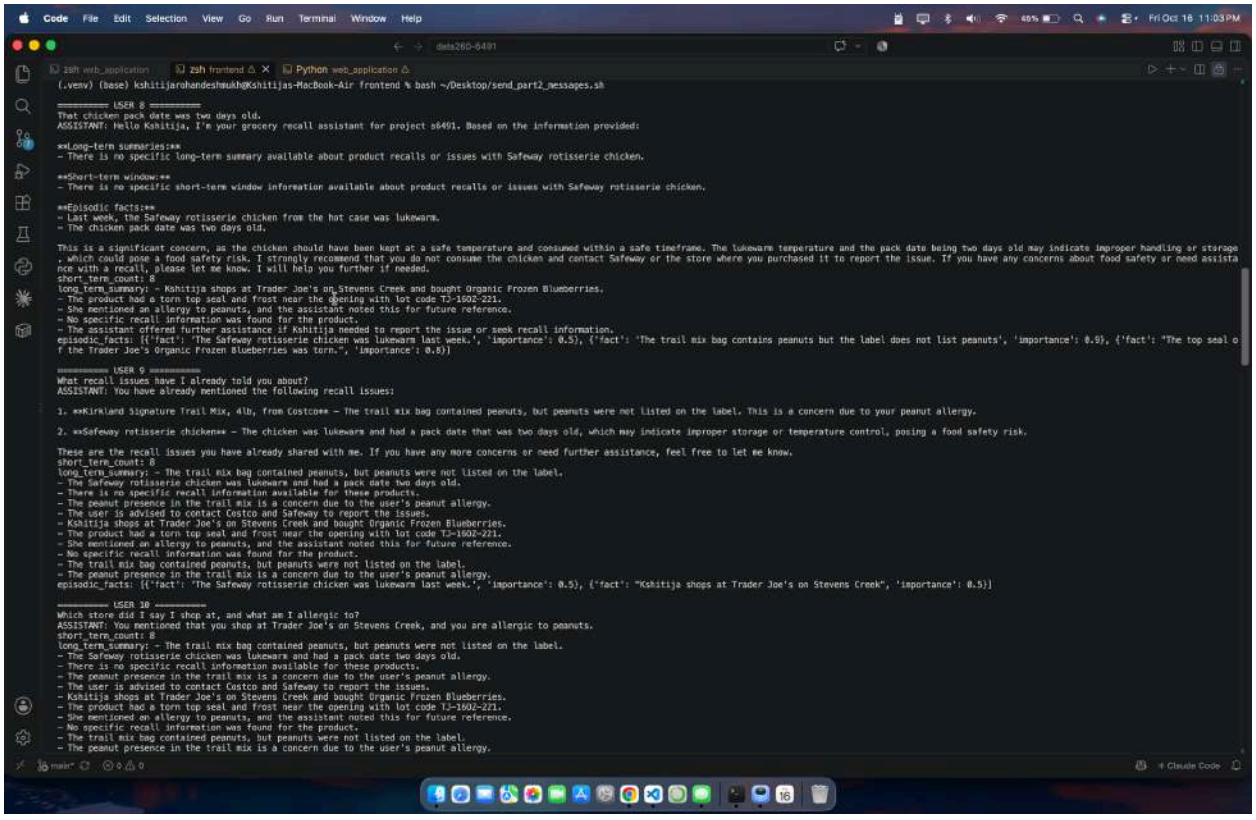
def extract_facts(client, message: str) -> list:
   raw = client.complete([
   {"role": "system", "content": EXTRACT_INSTRUCTIONS},
   {"role": "user", "content": message},
   ]).content
   return parse_json_object(raw).get("facts", [])[:3]

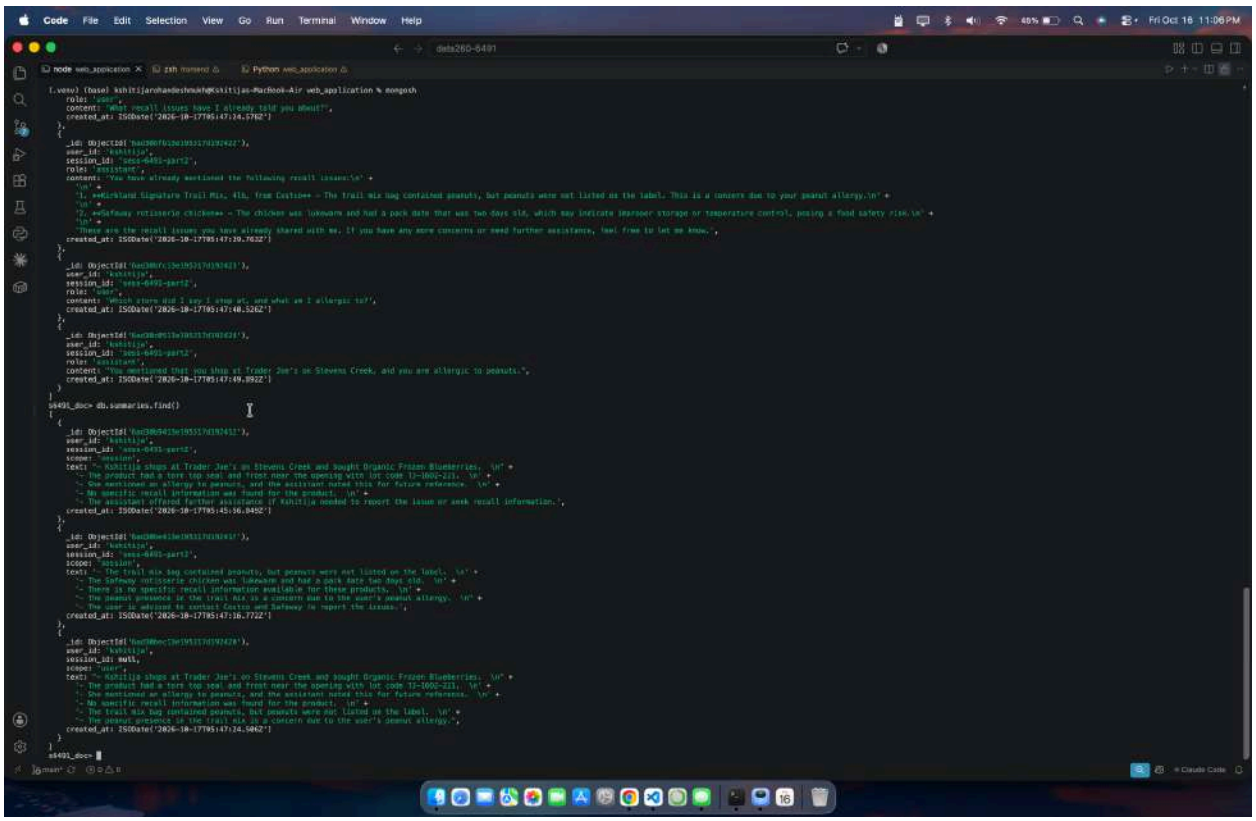
def retrieve_episodes(user_id: str, message: str, client, top_k: int) -> list:
   query_vector = embed_text(client, message)
   ranked = []
   for episode in get_episodes().find({"user_id": user_id}):
   ranked.append((cosine(query_vector, episode.get("embedding") or []), episode))
   ranked.sort(key=lambda pair: pair[0], reverse=True)
```

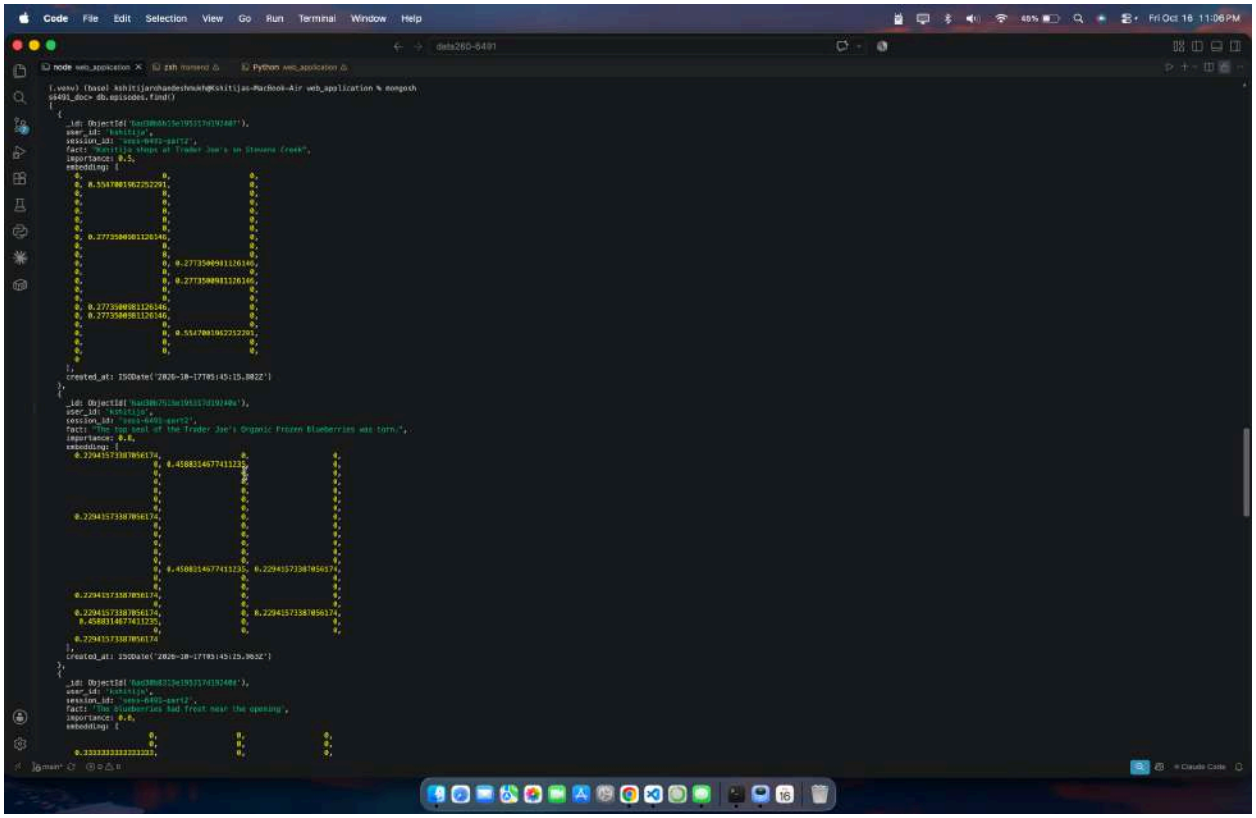
```
    return [{"fact": episode["fact"], "importance": episode["importance"]} for score,
episode in ranked[:top_k] if score > 0]

system = SYSTEM_PRIMER
if long_term_lines:
    system += "\n\nLong-term memory:\n" + "\n\n".join(long_term_lines)
if fact_lines:
    system += "\n\nEpisodic facts:\n" + "\n".join(fact_lines)

messages = [{"role": "system", "content": system}]
for item in prior_messages:
    messages.append({"role": item["role"], "content": item["content"]})
messages.append({"role": "user", "content": message})
```


(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air frontend % bash ~/Desktop/send_part2_messages.sh
========== USER 8 ==========
That chicken pack date was two days old.
ASSISTANT: Hello Kshitija, I'm your grocery recall assistant for project s6491. Based on the information provided:
Long-term summaries:
- There is no specific long-term summary available about product recalls or issues with Safeway rotisserie chicken.
Short-term window:
- There is no specific short-term window information available about product recalls or issues with Safeway rotisserie chicken.
Episodic facts:
- Last week, the Safeway rotisserie chicken from the hot case was lukewarm.
- The chicken pack date was two days old.
This is a significant concern, as the chicken should have been kept at a safe temperature and consumed within a safe timeframe. The lukewarm temperature and the pack date being two days old may indicate improper handling or storage, which could pose a food safety risk. I strongly recommend that you do not consume the chicken and contact Safeway or the store where you purchased it to report the issue. If you have any concerns about food safety or need assistance with a recall, please let me know. I will help you further if needed.
short_term_count: 8
long_term_summary: - Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The assistant offered further assistance if Kshitija needed to report the issue or seek recall information.
episodic_facts: [{'fact': 'The Safeway rotisserie chicken was lukewarm last week.', 'importance': 0.5}, {'fact': 'The trail mix bag contains peanuts but the label does not list peanuts', 'importance': 0.9}, {'fact': 'The top seal of the Trader Joe's Organic Frozen Blueberries was torn.', 'importance': 0.8}]
========== USER 9 ==========
What recall issues have I already told you about?
ASSISTANT: You have already mentioned the following recall issues:
1. **Kirkland Signature Trail Mix, 4lb, from Costco** - The trail mix bag contained peanuts, but peanuts were not listed on the label. This is a concern due to your peanut allergy.
2. **Safeway rotisserie chicken** - The chicken was lukewarm and had a pack date that was two days old, which may indicate improper storage or temperature control, posing a food safety risk.
These are the recall issues you have already shared with me. If you have any more concerns or need further assistance, feel free to let me know.
short_term_count: 8
long_term_summary: - The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The Safeway rotisserie chicken was lukewarm and had a pack date two days old.
- There is no specific recall information available for these products.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
- The user is advised to contact Costco and Safeway to report the issues.
- Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
episodic_facts: [{'fact': 'The Safeway rotisserie chicken was lukewarm last week.', 'importance': 0.5}, {'fact': "Kshitija shops at Trader Joe's on Stevens Creek", 'importance': 0.5}]
========== USER 10 ==========
Which store did I say I shop at, and what am I allergic to?
ASSISTANT: You mentioned that you shop at Trader Joe's on Stevens Creek, and you are allergic to peanuts.
short_term_count: 8
long_term_summary: - The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The Safeway rotisserie chicken was lukewarm and had a pack date two days old.
- There is no specific recall information available for these products.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.
- The user is advised to contact Costco and Safeway to report the issues.
- Kshitija shops at Trader Joe's on Stevens Creek and bought Organic Frozen Blueberries.
- The product had a torn top seal and frost near the opening with lot code TJ-1602-221.
- She mentioned an allergy to peanuts, and the assistant noted this for future reference.
- No specific recall information was found for the product.
- The trail mix bag contained peanuts, but peanuts were not listed on the label.
- The peanut presence in the trail mix is a concern due to the user's peanut allergy.


(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air web_application % mongosh
s6491_doc> db.summaries.find()



How to run and test
Send at least 10 messages in one session (via Postman/cURL) so the short-term window, summary generation, and episodic retrieval can all be observed. Optionally build a simple HTML page with one input box calling POST /api/chat to simulate a mini chat interface - not required, but recommended for screenshots.

Submit: a conversation screenshot showing the assistant's replies; the JSON result of GET /api/memory/{user_id}; the JSON result of GET /api/aggregate/{user_id}; MongoDB Compass showing the messages, summaries, and episodes collections with documents; and your code for the three endpoints plus the memory logic functions (episode extraction, summarization trigger, embedding + retrieval, prompt assembly).

Done as per above screenshots.

Part 3 - What Memory Costs

1. **Log memory usage**

For each /api/chat request, record:

- Prompt and completion tokens
- Tokens used by the system primer, long-term summary, short-term window, and episodic facts
- Request latency

Save the raw results as CSV or JSON.

```
log_chat_turn({
   "prompt_tokens": result.usage.input_tokens,
   "completion_tokens": result.usage.output_tokens,
   "primer_tokens": estimate_tokens(SYSTEM_PRIMER),
   "long_term_tokens": estimate_tokens(long_term),
   "short_term_tokens": estimate_tokens(short_term_text),
   "episodic_tokens": estimate_tokens(episodic_text),
   "latency_ms": int((time.perf_counter() - started) * 1000),
})
```

2. **Create memory probes**

Before testing, create memory_probes.yaml with five fixed questions, expected answer criteria, and a 1–5 scoring rubric.

```
rubric:
 5: All expected facts are present and specific.
 1: None of the expected facts are present.
probes:
 - id: name_store
   question: What is my name, and which store location do I shop at?
   expected: ["Kshitija", "Stevens Creek"]
```

3. **Compare short-term memory sizes**

Run the same 30-message conversation in fresh sessions with:

- SHORT_TERM_N = 4
- SHORT_TERM_N = 12
- SHORT_TERM_N = 30

Ask the same five probe questions after each run. Report the mean prompt tokens per turn, mean latency, and mean probe-quality score.

4. **Test one memory-saving policy**

Implement one eviction or compression policy. Repeat the conversation and probes, then report token savings and any change in probe quality.

3 and 4 ->

```
def short_term_n() -> int:
    return int(os.getenv("SHORT_TERM_N", "8"))


def apply_memory_policy(prior: list, episodes: list) -> tuple:
    if memory_policy() != "compress":
        return prior, episodes
    if len(prior) > 4:
        older = " | ".join(item["content"].replace("\n", " ")[:40] for item in
prior[:-4])
        prior = [{"role": "user", "content": "Compressed earlier turns: " + older}] +
prior[-4:]
    return prior, episodes[:1]
```

5. **Estimate memory cost**

Using input pricing of $1 per million tokens and output pricing of $5 per million tokens, estimate the cost of 10,000 30-turn conversations under your best and worst settings.

```
def cost_for_10000(prompt_mean: float, completion_mean: float) -> float:
    turns = 30 * 10000
    return (prompt_mean * turns) / 1_000_000 * 1 + (completion_mean * turns) /
1_000_000 * 5
```

No paid hosted-model run is required. **Explain how you selected the best and worst settings. Submit the memory-cost table, memory_probes.yaml, and the eviction/compression comparison results.**

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1 - 5)
SHORTTERMN=4	614.8	17577	3.4
SHORTTERMN=12	979.2	26304	4.6
SHORTTERMN=30	1572	32529	4.2
Eviction/compression policy	774.5	27085	4.2

Best setting: SHORT_TERM_N=12.
Worst setting: SHORT_TERM_N=30.

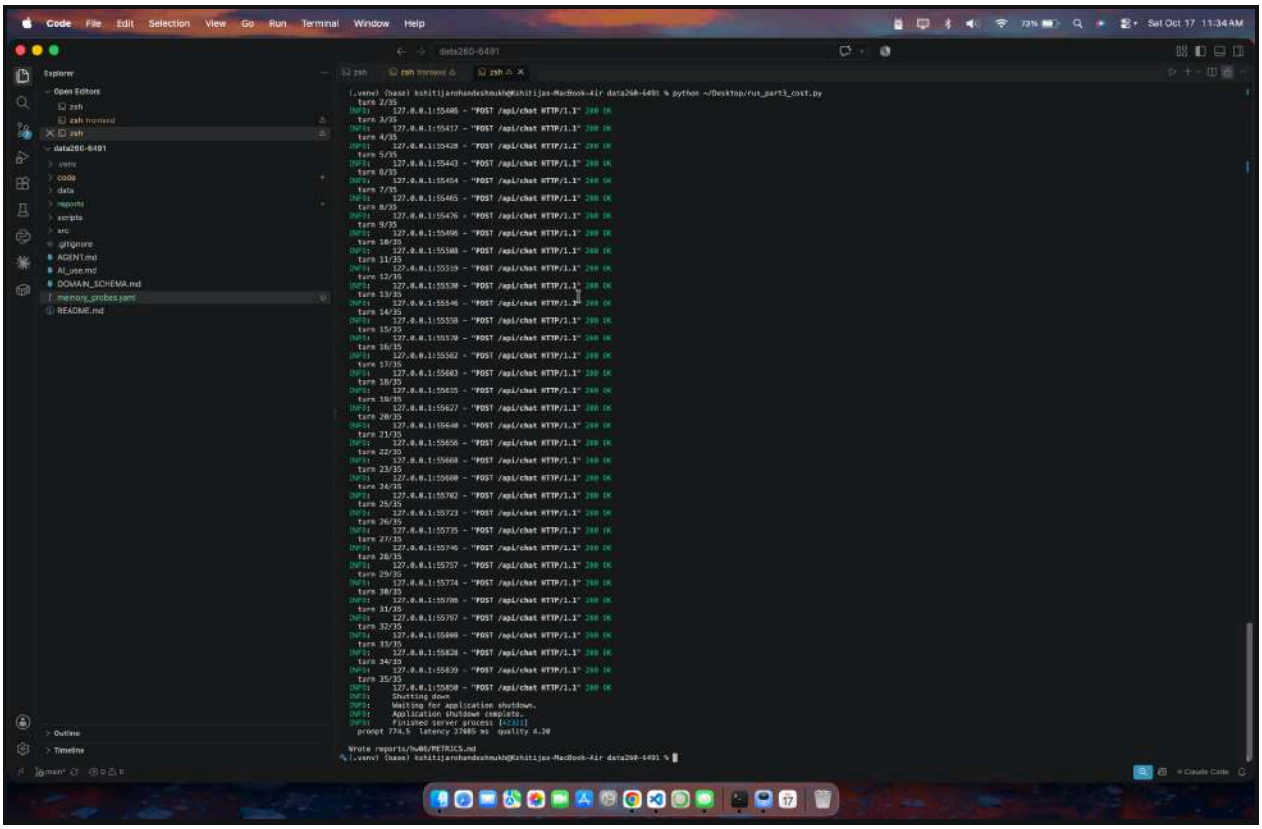
The best setting is the one with the highest probe score. If two scores are close, the lower prompt-token setting is preferred. The worst setting is the one with the highest mean prompt tokens.

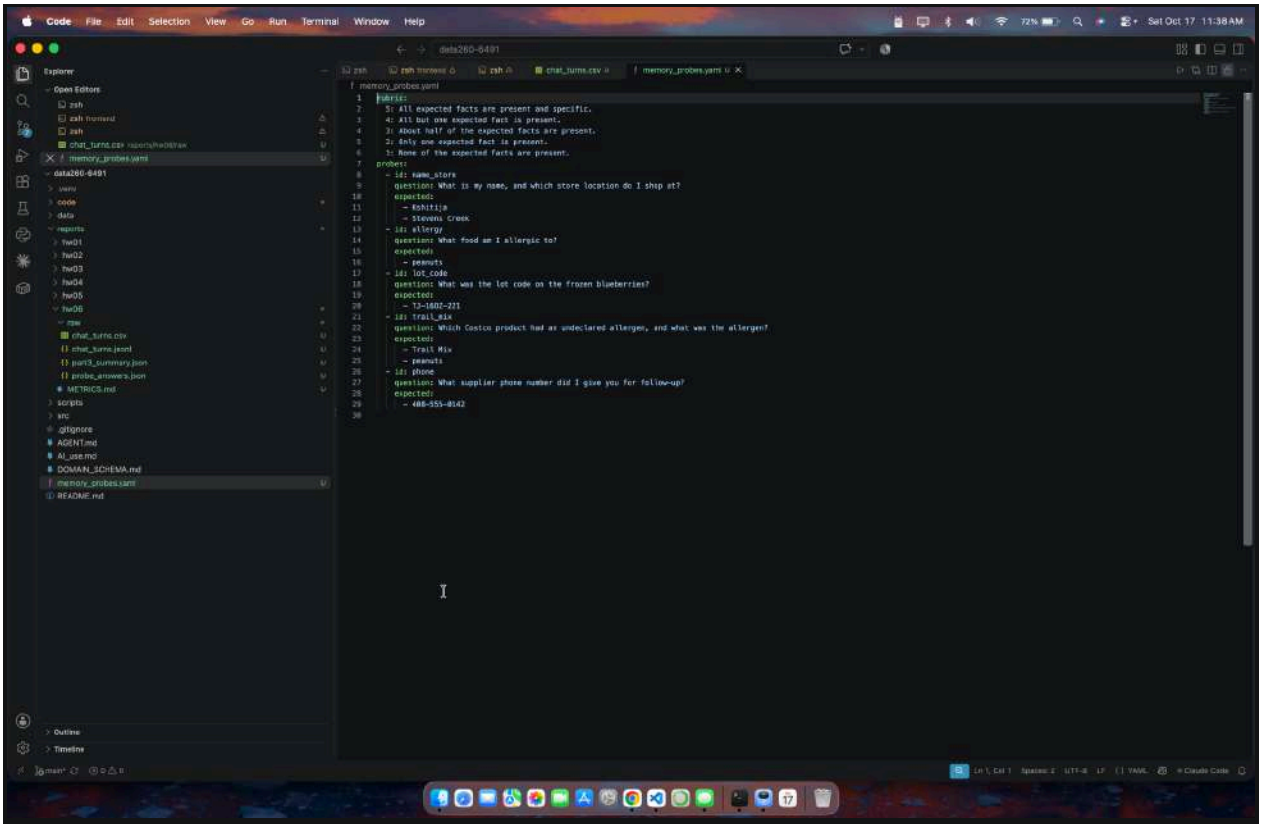
Cost of 10,000 conversations of 30 turns, at $1 per million input tokens and $5 per million output tokens:

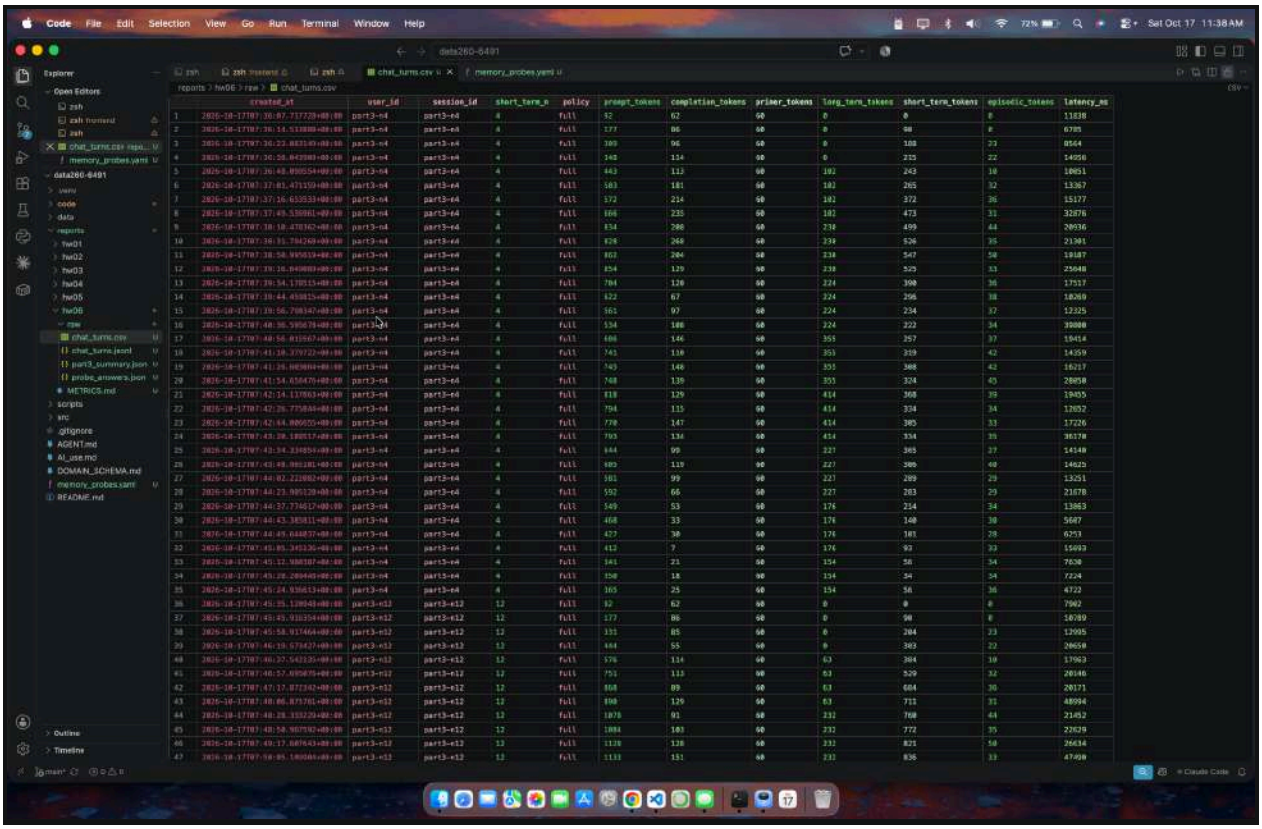
- Best (SHORT_TERM_N=12): $445.02
- Worst (SHORT_TERM_N=30): $602.49

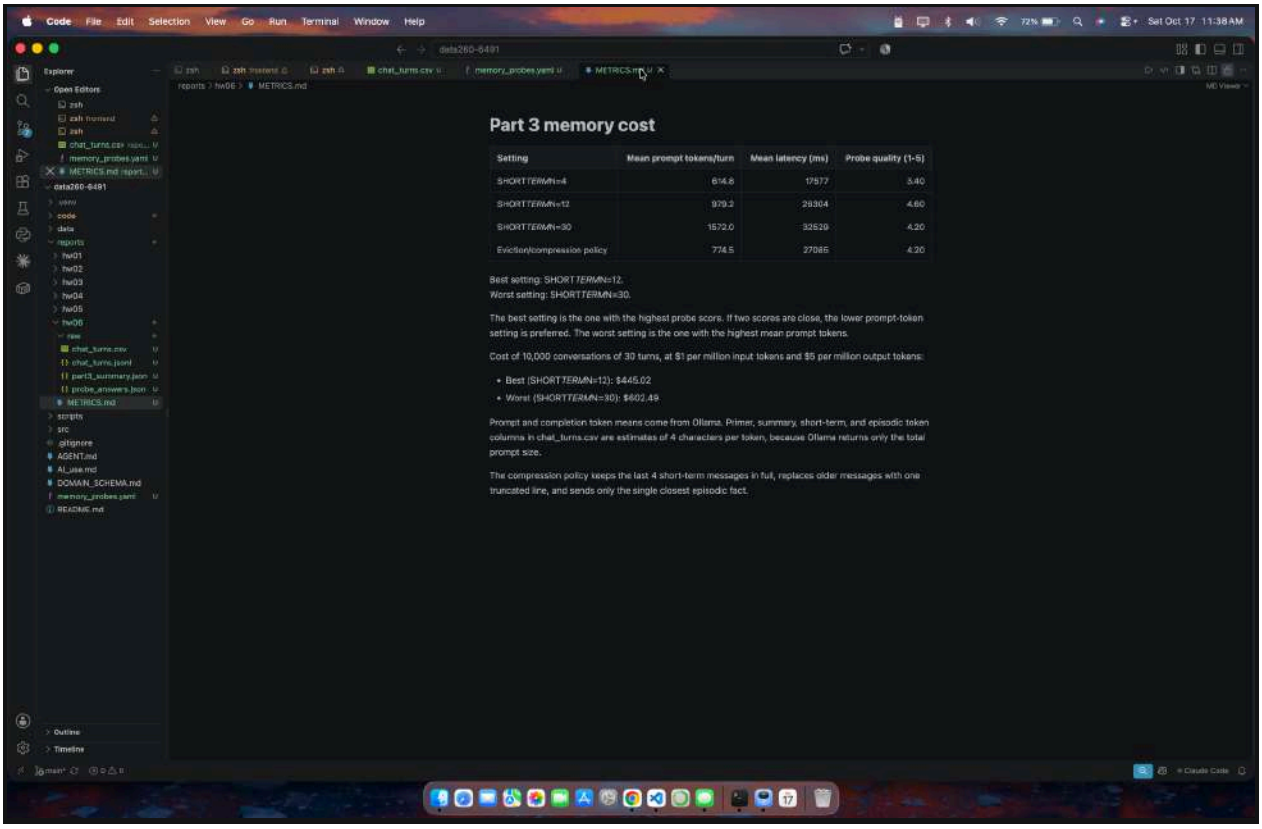
Prompt and completion token means come from Ollama. Primer, summary, short-term, and episodic token columns in chat_turns.csv are estimates of 4 characters per token, because Ollama returns only the total prompt size.

The compression policy keeps the last 4 short-term messages in full, replaces older messages with one truncated line, and sends only the single closest episodic fact.









Part 3 memory cost

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	614.8	17577	3.40
SHORT_TERM_N=12	979.2	28304	4.60
SHORT_TERM_N=30	1572.0	32529	4.20
Eviction/compression policy	774.5	27085	4.20

Best setting: SHORT_TERM_N=12.
Worst setting: SHORT_TERM_N=30.

The best setting is the one with the highest probe score. If two scores are close, the lower prompt-token setting is preferred. The worst setting is the one with the highest mean prompt tokens.

Cost of 10,000 conversations of 30 turns, at $1 per million input tokens and $5 per million output tokens:

- Best (SHORT_TERM_N=12): $445.02
- Worst (SHORT_TERM_N=30): $602.49

Prompt and completion token means come from Ollama. Primer, summary, short-term, and episodic token columns in chat_turns.csv are estimates of 4 characters per token, because Ollama returns only the total prompt size.

The compression policy keeps the last 4 short-term messages in full, replaces older messages with one truncated line, and sends only the single closest episodic fact.

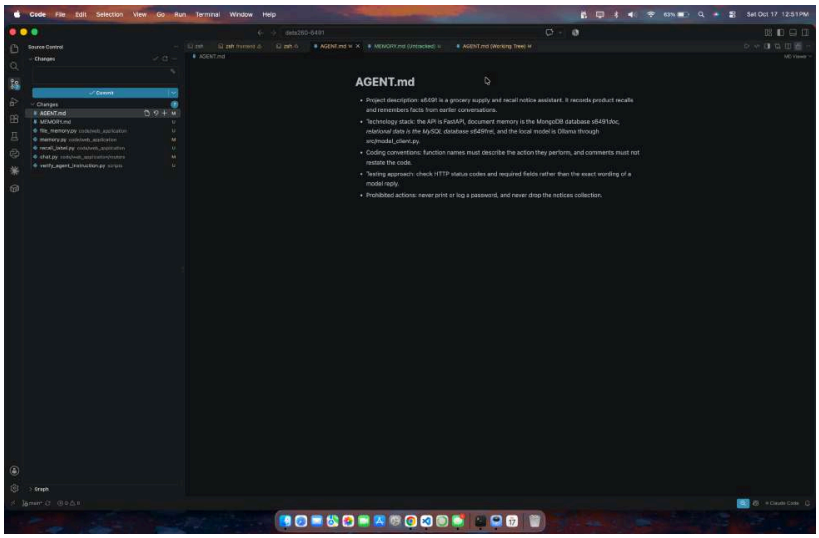
Part 4 - File-Based Memory and Consolidation

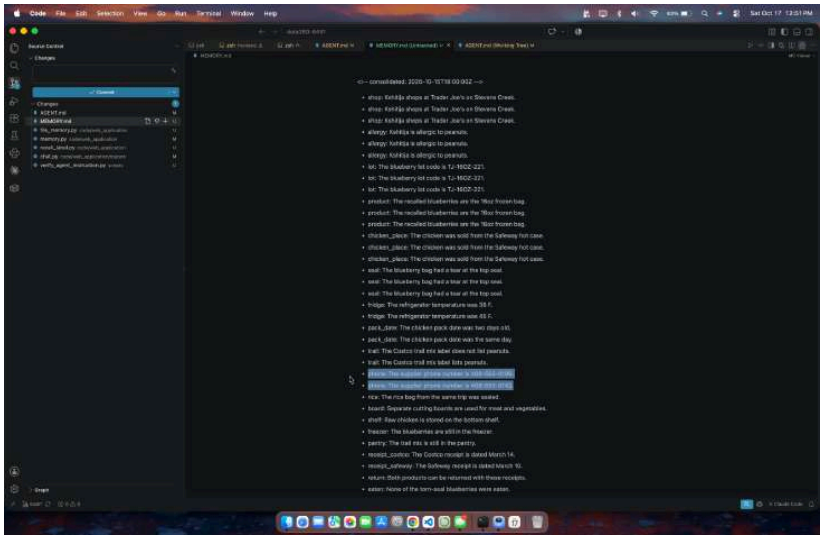
In this part, you will build a second memory system using AGENT.md and MEMORY.md, then compare it with your MongoDB-based memory system.

1. Create the memory files

Reuse and expand the AGENT.md file from HW1.

- AGENT.md stores durable project and agent instructions.
- MEMORY.md stores durable facts learned from previous conversations.





2. Implement file-based memory

Create functions that:

- Load both Markdown files
- Add their contents to the system prompt
- Read the last-consolidated timestamp
- Determine when memory consolidation is required

Consolidate MEMORY.md when:

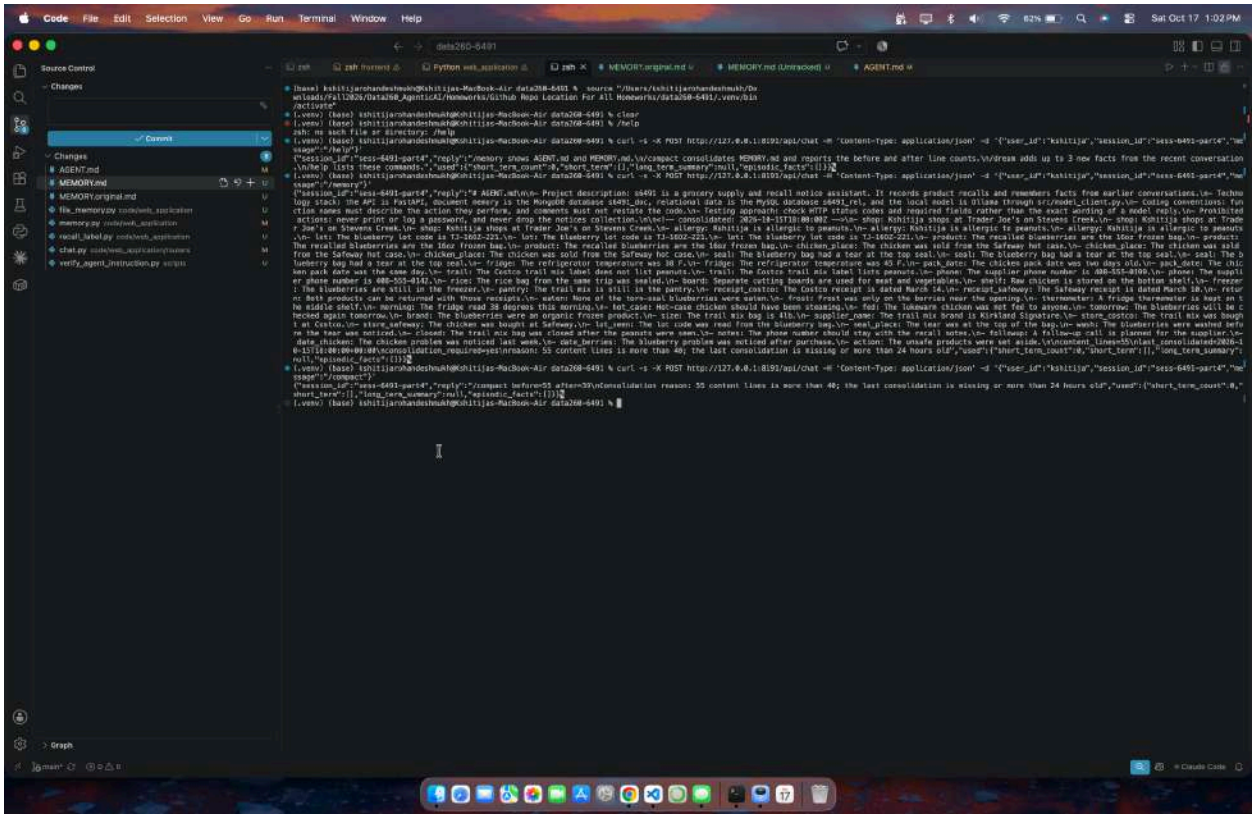
- It contains more than 40 non-empty, non-comment lines, or
- Its recorded consolidation time is more than 24 hours old

```
def file_memory_block() -> str:
    agent = read_text(agent_path()).strip()
    memory = read_text(memory_path()).strip()
    return f"File memory instructions:\n{agent}\n\nDurable facts:\n{memory}"
```

```
def consolidation_required(text: str) -> tuple:
    count = len(content_lines(text))
    recorded = last_consolidated_at(text)
    too_long = count > LINE_LIMIT
    too_old = recorded is None or utcnow() - recorded > STALE_AFTER
```

```
    from file_memory import file_memory_block


    system = SYSTEM_PRIMER + "\n\n" + file_memory_block()
```



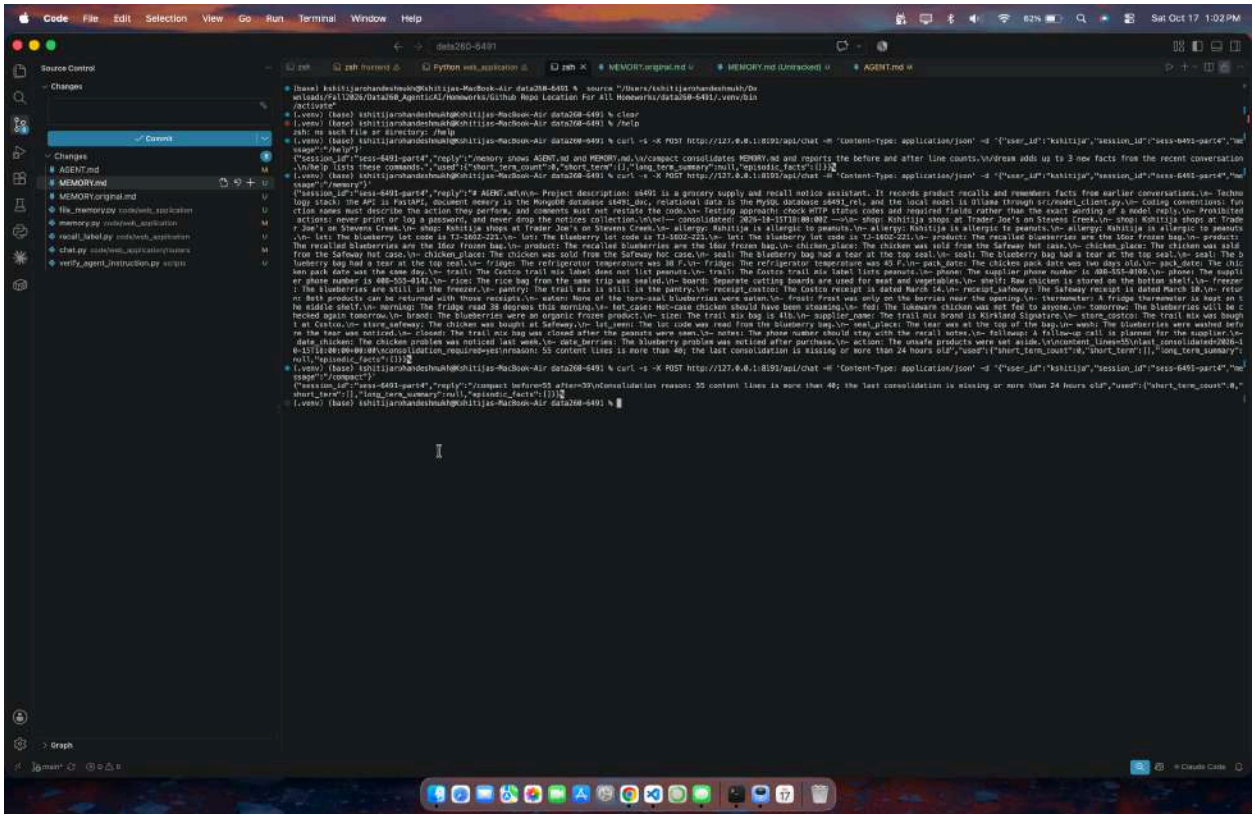
3. Support memory commands

Implement the following commands:

- /memory — display the current file-based memory
- /compact — consolidate MEMORY.md and report the message count before and after
- /dream — identify useful long-term facts from recent conversation and add them to MEMORY.md
- /help — explain the available commands

```python
def post_chat(body: ChatIn):
    message = body.message.strip()
    session_id = body.session_id or f"sess-{uuid.uuid4().hex[:8]}"
    if message.startswith("/"):
        return run_memory_command(message, body.user_id, body.session_id or "")
```

```python
def run_memory_command(command: str, user_id: str, session_id: str) -> dict:
    name = command.strip().split()[0]
    if name == "/help":
        ...
    elif name == "/memory":
        ...
    elif name == "/compact":
        reply = compact_memory()
    elif name == "/dream":
        reply = remember_recent_facts(user_id, session_id)
```

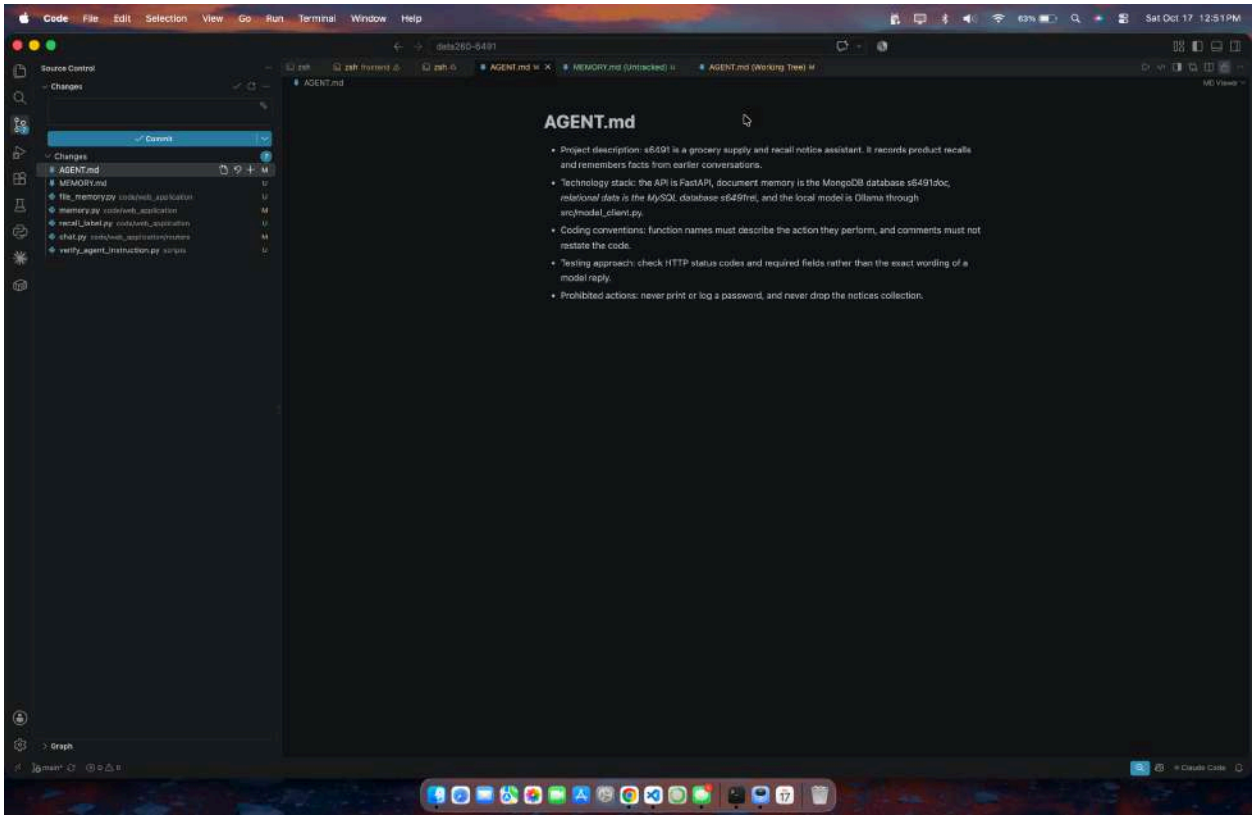


4. Expand and test AGENT.md

By the end of HW6, AGENT.md must contain at least five instructions covering at least three of these categories:

- Project description
- Technology stack
- Coding conventions
- Testing approach
- Prohibited actions Generate or implement one function and verify that it follows the instructions in AGENT.md.

```python
def format_recall_notice(product_name: str, recall_type: str) -> str:
    return f"{product_name} — {recall_type}"
```






zsh
zsh frontend
Python web_application
zsh
MEMORY.original.md U
file_memory.py U
MEMORY.md (Untracked) U
AGENT.md M
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 % curl -s -X POST http://127.0.0.1:8191/api/chat -H 'Content-Type: application/json' -d '{"user_id":"kshitija","session_id":"sess-6491-part4","message":"/dream"}'
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 %

5. Test consolidation

Create a 55-line MEMORY.md containing:

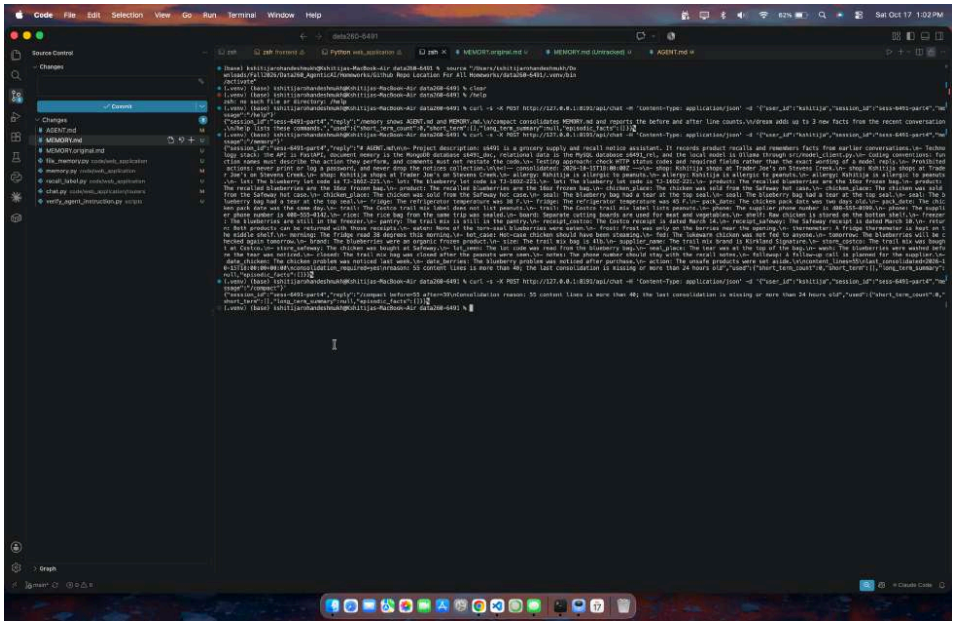
- At least three duplicated facts
- At least three deliberately contradictory facts

Run your consolidation process and reduce the file to fewer than 40 lines.
Your consolidation must:

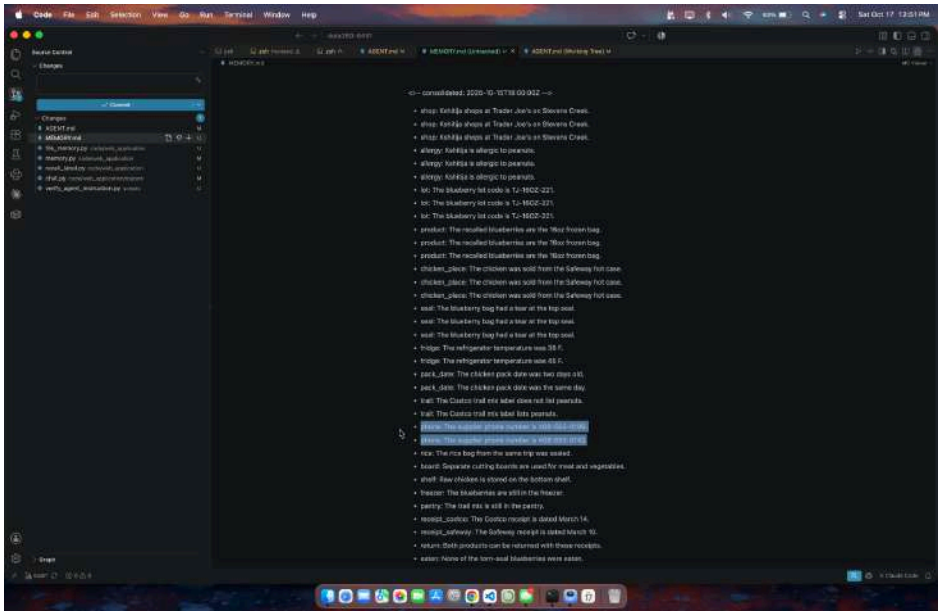
- Merge duplicate facts
- Resolve contradictions explicitly
- Avoid silently deleting conflicting information
- Identify any contradiction that was resolved incorrectly Commit both the original and consolidated versions to Git.

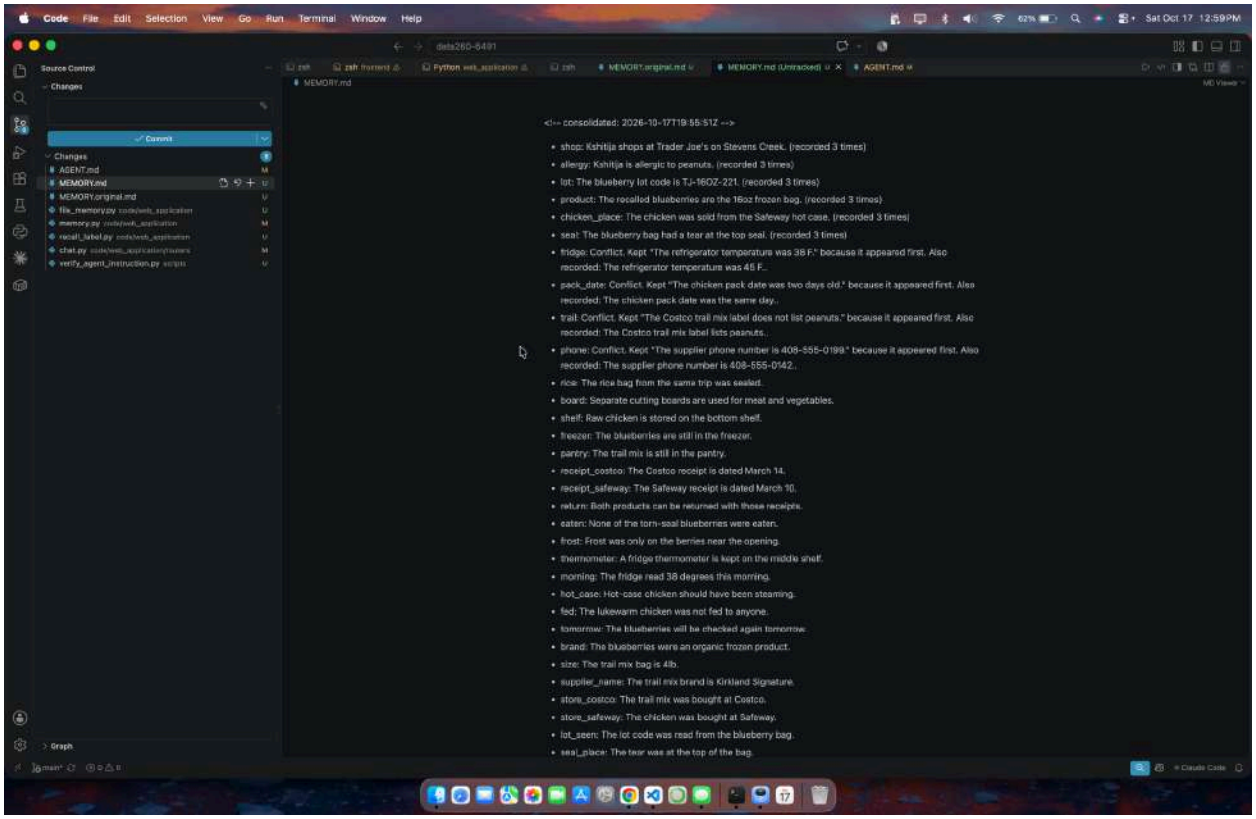
The /compact command reduced MEMORY.md from 55 content lines to 39 content lines (before=55, after=39).

```
        if len(unique) == 1:
            suffix = f" (recorded {len(values)} times)" if len(values) > 1 else ""
            written.append(f"- {key}: {unique[0]}{suffix}")
        else:
            rest = "; ".join(unique[1:])
            written.append(
                f"- {key}: Conflict. Kept \"{unique[0]}\" because it appeared first.
Also recorded: {rest}."
            )
```



The phone contradiction was resolved incorrectly. Consolidation kept 408-555-0199 because it appeared first. The correct number is 408-555-0142. The other number is still written on the same line.



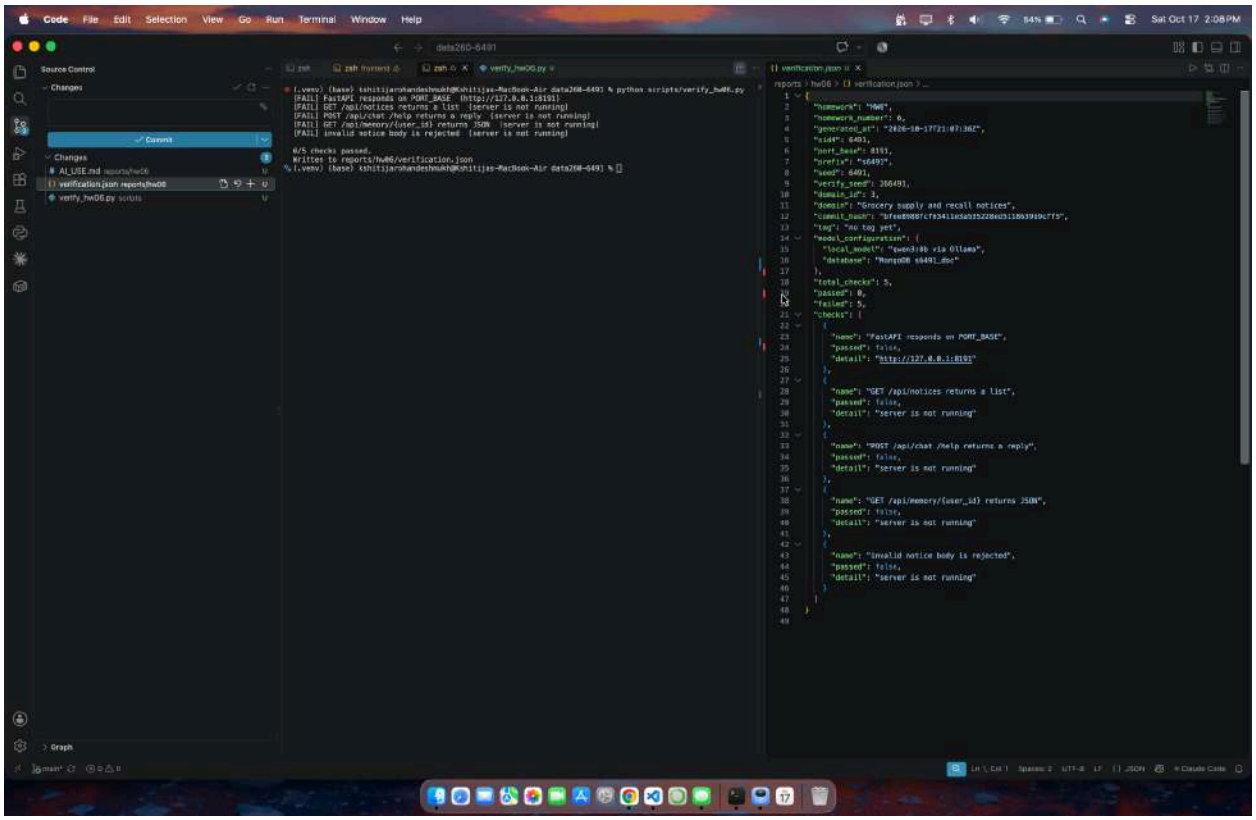


6. Compare both memory systems

Compare MongoDB-backed memory and file-based memory in terms of:

- Inspectability
- Persistence after a restart
- Scalability across many sessions
- Suitability for facts that must not be forgotten

1. MongoDB memory is useful when there are many users and conversations. We can inspect it by searching the messages, summaries, and episodes stored for each user or session. File-based memory is easier to inspect because we can simply open MEMORY.md and read it.
2. Both systems keep their data after the application restarts. MongoDB stores its data in the “s6491_doc” database, while file-based memory stores information directly on the computer.
3. MongoDB is better for handling many sessions because each user's data is stored separately. A single Markdown file can become large and difficult to organize.
4. File-based memory is better for important facts and instructions that should always be remembered. MongoDB may store a fact but not include it in a response if that fact is not considered relevant to the current conversation.



AI_USE.md

1. What did you use an AI assistant for, and what did you do yourself?

I used an AI assistant when I got stuck on the memory parts. It helped me draft the MongoDB, the file-based commands, the consolidation logic. It also helped me to resolved the errors I got.

What I did myself: I set up the repo, set up MongoDB, and kept the FastAPI server and Ollama running. I sent the curl requests, looked at the collections in mongosh, and took the screenshots. I filled the Part 3 table from METRICS.md and checked the report against the assignment.

2. One AI-produced output that was wrong or unsuitable

The server command I was given was uvicorn code.web_application.main:app while I was already inside the web_application folder. It looked like the normal start command, but the server never came up.

3. How I detected the problem

I ran it and read the traceback. It ended with ModuleNotFoundError: No module named 'code.web_application'; 'code' is not a package. Python was picking up its own code module instead of my project folder, so the app never started.

4. What was changed and why it works now

From the web_application folder I started it with uvicorn main:app --host 127.0.0.1 --port 8191. The server stayed up after that, and the notice and chat requests started returning normal HTTP responses.